

実験ログ: 液体と MPM (FLIP・砂・雪)

2026-09-24 | 収録 22 件

Houdini 研究部で、hython で実際に測った実験の記録。数字はどれも測った値。

収録している実験

- ・ FLIP 液体 — 組み方が分からず保留
- ・ FLIP 液体 — 配線が解けた。そして粒の数は体積ではなかった
- ・ 液体に表面を張る — そして019の係数が間違っていた
- ・ MPM に入る — 落ち方は式に乗る。ただし時刻が 0.0742フレーム先に進んでいる
- ・ MPM でぶつける — 砂は 16% 膨らみ、既定の塊は 31% 縮む
- ・ MPM の粒を面に戻す — 27% 太る。膨らむ厚みは粒の間隔の 0.4倍
- ・ MPM の刻みを細かくする — 量によって、落ち着く速さが違った
- ・ MPM の粒を細かくする — 刻みより粒のほうが効くし、しかも安い
- ・ MPM の風 — 風速だけ上げて何も起きない。抵抗は速さの2乗だった
- ・ MPM の地面の摩擦 — 教科書の式に 0.04% で乗る。ただし塊が潰れるまで
- ・ MPM の材質5つで滑らせる — 摩擦の式に乗るのは、形が崩れない材質だけ
- ・ MPM のコライダの摩擦 — 式は同じ。ただし地面と重ねると完全に無視される
- ・ 動く板は塊をどれだけ運ぶか — 追いつくまでは式どおり。追いついた先で潰れて遅れる
- ・ 回る台は塊をどこまで乗せておけるか — 境目は式の形どおり。ただし摩擦は 0.3~0.4 ぶんしか効かない
- ・ たわむ板は塊を放り上げられるか — Deforming は板の速さを渡す。Rigid はたわみを平均の動きにしてしまう
- ・ 液体はいつ暴れ出すか — 摩擦の境目ではなく「時間」だった。substep 8 で消えるが 5倍かかる
- ・ sticky は天井にぶら下げられるか — 地面とコライダで効き方は同じ。ただし重さに逆らう力は無い
- ・ 摩擦の上限は、粒の細かさで上がる — substep を 8 にしても1ミリも動かない
- ・ 粒を細かくしても、摩擦の式には戻らない — 細かいほど塊が潰れ、かえって式から離れる
- ・ 液体の暴れは、粒を細かくしても消えない — 飛ぶ高さは下がるが、始まる時刻は変わらない
- ・ mpmsource は箱に「体積 ÷ Particle Separation³」個の粒を詰める — pscale には Separation そのものが入る
- ・ FLIP のダムブレイクは Particle Separation 0.04 で足りる — 0.06 より粗いと壁ぎわの数粒が大きな塊に見え、水も 1~2 割増える

FLIP 液体 — 組み方が分からず保留

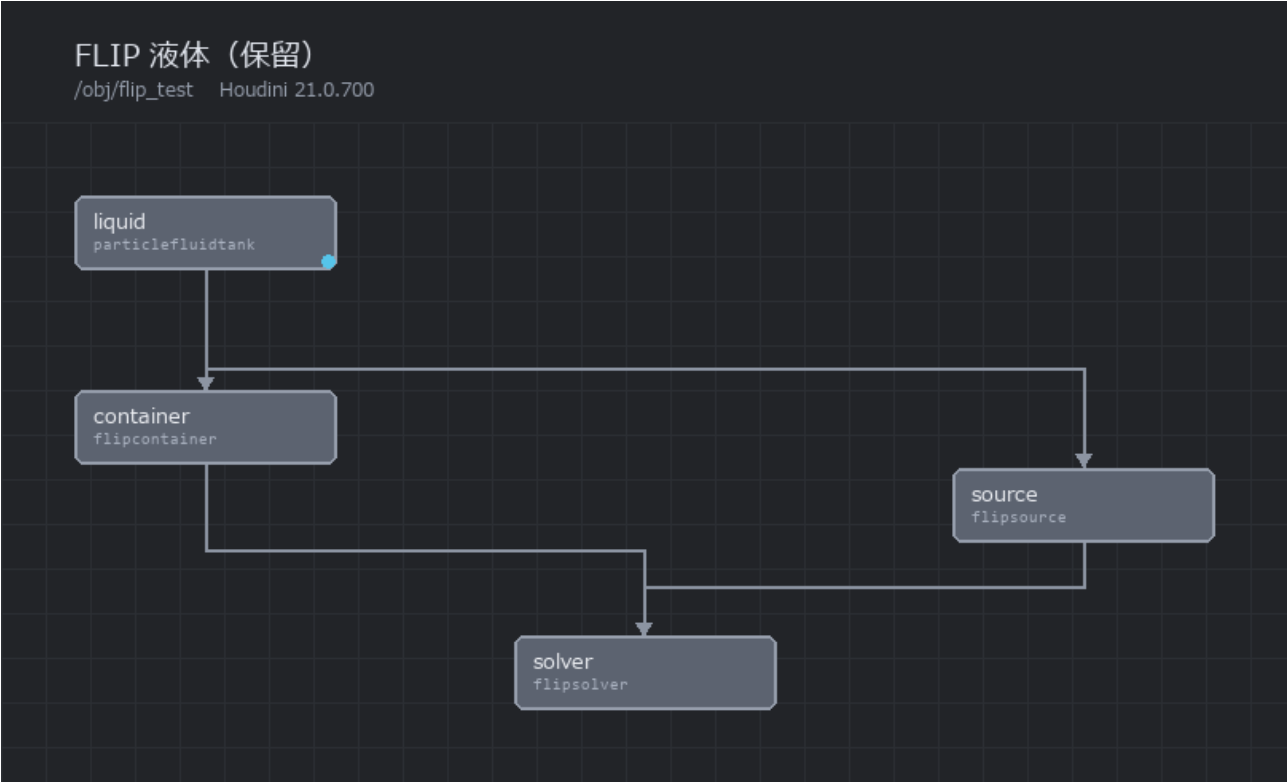
2026-09-24 | Houdini 21.0.700 | /obj/flip_test | ノード 4 / 接続 4

何を作ったか

液体は押し縮められないので体積が保たれるはず。FLIP は液体を粒で表す方式なので、粒1つが一定の体積を担うなら「粒の数」がそのまま体積にあたる。さらに FLIP には reseeding という粒を足したり間引いたりする仕組みがあり、これを切れば粒の数が完全に保たれるはずーという筋書きで実験を組んだ。

ところがソルバが動かなかった。4通りの配線を試したがすべて同じエラーで止まり、原因を特定できていない。ここまでの調査内容を残して保留とする。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
liquid	particlefluidtank	particlesep = 0.15 sizex = 2.0 sizey = 2.0 sizez = 2.0
container	flipcontainer	particlesep = 0.15
source	flipsource	particlesep = 0.15

ノード	タイプ	デフォルトから変更したパラメータ
solver	flipsolver	folder14 = 1 folder15 = 1 folder2 = 1 folder3 = 1 folder5 = 1 folder6 = 1 particlesep = 0.15 useground = 1 vis_ramp1cb = 1.0 vis_ramp2cb = 1.0 vis_ramp2cg = 0.5 vis_ramp2pos = 0.4993646740913391 …ほか 5 件

試した配線と結果

すべて flipsolver が「Not enough sources specified.」というエラーを返した。出力2には入力の粒がそのまま出てくるが、出力1（本来の結果）は空のまま。

配線	出力1	出力2	出力3	エラー
タンクの粒 → 入力1	なし	0点	0点	Not enough sources specified.
コンテナ → 入力1	なし	0点	0点	Not enough sources specified.
コンテナ → 入力1、粒 → 入力2	なし	1460点（素通し）	0点	Not enough sources specified.
コンテナ → 入力1、flipsource → 入力2	なし	1点（素通し）	0点	Not enough sources specified.

分かっていること

調べたこと	結果
particlefluidtank で粒は作れる	2×2×2、間隔0.15 で 1460粒
flipcontainer は単体で動く	出力1が7点、出力2が82点、出力3が2点。エラーなし
flipsource は単体で動く	1点1プリミティブ（ポリウム）。エラーなし
flipsolver の入力数	4つ（input1～input4）
flipsolver の出力数	3つ
source という名前のパラメータ	sourceband（既定 2.0）の1つだけ

わかったこと

- ・エラーメッセージは一貫して「Not enough sources specified.」。ソルバは入力のどこかに「液体の供給源」を期待しているが、粒を渡しても、コンテナを渡しても、flipsource を渡しても満たされなかった。
- ・他のソルバとの違いが大きい。rdbbulletsolver（実験014）も vellumsolver（実験015）も pyrosolver（実験016）も、入力1にジオメトリをつなぐだけで動いた。FLIP だけが追加の作法を要求している。

- ・パラメータ側にも手がかりが少ない。source を含む名前は sourceband ひとつしかなく、「どの入力供給源か」を指定する仕組みは見当たらない。つまり入力の順番や種類が決まっていて、そこに正しいものを渡す必要があるということ。

- ・途中で1つ罌を踏んだので記録しておく。particlefluidtank を上に移動させたら粒が0になった。このノードは「水位」を基準に中身を満たす仕組みで、タンクを水位より上に動かすと空になる。落下させたいときはタンクを動かすのではなく、地面のほうを下げる。

- ・この保留は失敗ではあるが、記録しておく価値がある。「他と同じやり方が通じないノードがある」と分かったこと自体が情報で、次に取り組むときに同じ4通りを試さずに済む。

- ・なお粒から見積もった体積は 4.92750 で、箱の体積 8.0 の 62%

だった。粒は格子状に並ぶので隙間があり、粒の数 ×

間隔の3乗では体積を正確には表せない。実際に検証に使うなら、この係数を先に求めておく必要がある。

次にやること

- ・Houdini 付属のヘルプ (nodes.zip) で flpsolver

の入力の説明を読む。ネットではなく手元の一次情報から

- ・シェルフツールが作る FLIP のネットワークを GUI で作り、その構造を .hipnc から読み取る

- ・先に Karma でのレンダリングを片付ける。実験016でボリウムが描けないと分かっているため

FLIP 液体 —

配線が解けた。そして粒の数は体積ではなかった

2026-09-24 | Houdini 21.0.700 | /obj/flip | ノード 5 / 接続 6

何を作ったか

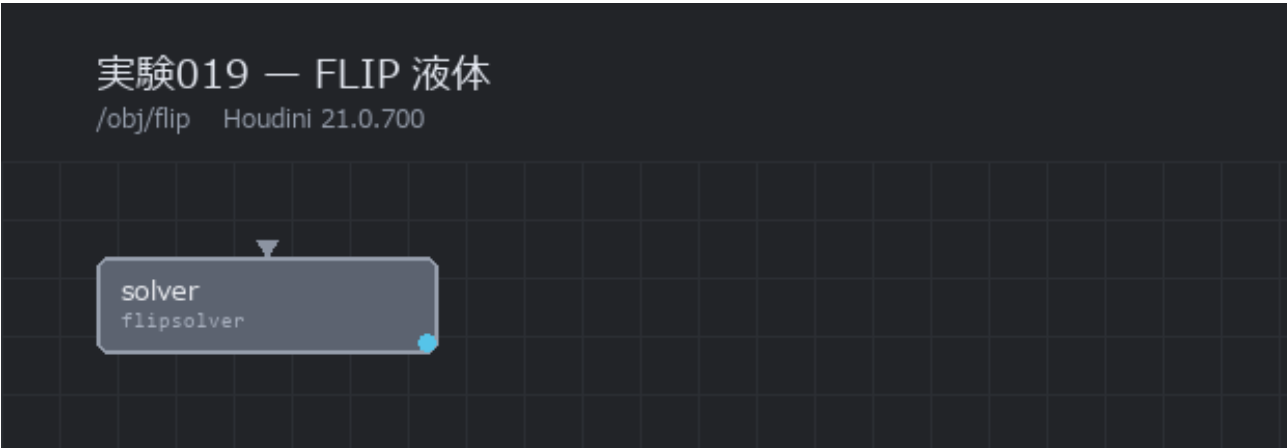
実験017で4通りの配線を試して全部同じエラーで止まり、保留にしていた。付属ヘルプ（nodes.zip）を読んだら原因がはっきりした。

コンテナは単なる箱ではなく、3つのチャンネル（液体の粒・容器・衝突）を束ねる中継点だった。粒はソルバに直接つながるのではなく、いったんコンテナに入れて、コンテナの3出力をソルバの3入力へ渡す。017で試した4通りは、どれもこの形になっていなかった。

さらに調べると、そのチャンネルを流れているのは粒ではなく名前付きのVDBで、source という名前のものだけが液体を湧かせると分かった。

動くようになってから本題に入った。液体は押し縮められないので体積が保たれる。粒1つが一定の体積を担うなら「粒の数」が体積の代わりに使えるはずーという筋書きは、半分外れた。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
container	flipcontainer	particlesep = 0.07 sizex = 3.0 sizey = 3.0 sizez = 3.0
blob	box	sizex = 0.9 sizey = 0.9 sizez = 0.9 ty = 1.0
src	flipsource	particlesep = 0.07 volumename = source
merge_sources	merge	(変更なし)

ノード	タイプ	デフォルトから変更したパラメータ
solver	flipsolver	donarrowband = 0 folder14 = 1 folder15 = 1 folder2 = 1 folder3 = 1 folder5 = 1 folder6 = 1 ground_posy = -1.5 particlesep = 0.07 useground = 1 vis_ramp1cb = 1.0 vis_ramp2cb = 1.0 …ほか 7 件

017で試した配線と、正解

flipsolver の入力 は 1=Sources（液体の粒）、2=Container、3=Collisions、4=Boundary Flow。ヘルプにそう書いてある。ただしそれだけでは足りず、コンテナ自身が同じ3チャンネルの入出力を持つ中継点であることが鍵だった。

配線	結果
017(a) 粒 → ソルバ入力1	Not enough sources specified.
017(b) コンテナ → ソルバ入力1	Not enough sources specified.
017(c) コンテナ → 入力1、粒 → 入力2	Not enough sources specified.
017(d) コンテナ → 入力1、flipsource → 入力2	Not enough sources specified.
019 粒 → ソルバ入力1、コンテナ → 入力2	エラーは消えたが粒が流れない
019 粒 → コンテナ入力1、コンテナの3出力 → ソルバの3入力	動いた

Sources チャンネルを流れているもの

コンテナの出力1を調べると、点は7つでプリミティブは7つ、すべてVDB（ボリューム）だった。ヘルプには「液体の粒を期待する」と書いてあるが、実際に流れているのは粒ではない。だからタンクの3,145点を入れても何も起きなかった。

VDBの名前	役割の見当
source	液体が湧く場所
sink	液体が消える場所
velsurface	速度を与える面
guidingsurface	流れを誘導する面
pressuresurface	圧力を与える面
vel	速度の場
pressure	圧力の場

VDBの名前を変えると液体は湧くか

同じ箱から flipsource でVDBを作り、名前だけを変えてコンテナの Sources チャンネルに合流させた。flipsource の既定の名前は surface

なので、そのままつないでも何も起きない。

VDBの名前	F1	F5	F10	F20	F40	判定
source	3588	3667	4913	5725	5517	液体が湧く
surface (flipsource の既定)	5	5	5	5	5	何も起きない
notasource	5	5	5	5	5	何も起きない

水位を変えたときの粒の数 — narrow band の有無

コンテナは 3×3×3 で原点中心。水位 −1.0 のとき深さ0.5、+1.0 のとき深さ2.5になる。水の体積は5倍になる。

水位	水の体積	narrow band あり	narrow band なし
−1.00	4.5000	19,973	33,252
−0.50	9.0000	19,081	48,926
0.00	13.5000	19,537	64,517
0.50	18.0000	19,885	79,896
1.00	22.5000	19,484	94,621

narrow band を切ったときの当てはまり

粒の数 = 3415.7 × 体積 + 18130 という直線に当てはめた結果。

水の体積	実測	直線の予測	ずれ
4.500	33,252	33,500.8	−0.74%
9.000	48,926	48,871.6	+0.11%
13.500	64,517	64,242.4	+0.43%
18.000	79,896	79,613.2	+0.36%
22.500	94,621	94,984.0	−0.38%

40フレーム進めたときの粒の数（水位 0.0、narrow band なし）

017では「reseeding を切れば粒の数が完全に保たれるはず」と書いていた。実測は逆だった。

reseeding	F1	F5	F10	F20	F30	F40	最大-最小
あり（既定）	64,517	64,500	64,439	64,421	64,408	64,385	132 (0.20%)
なし	64,517	66,906	69,929	75,904	81,898	87,921	23,404 (36.28%)

わかったこと

・017を止めていた原因は、コンテナの役割の誤解だった。flipcontainer は箱を作るだけのノードではなく、液体の粒・容器・衝突の3チャンネルを束ねて流す中継点になっている。粒はソルバに直接つながるのではなく、コンテナに入れてから、コンテナの3出力をソルバの3入力へ渡す。

- ・そのチャンネルを流れているのは粒ではなく名前付きのVDBだった。コンテナの出力1は7つのVDBで、名前は source / sink / velsurface / guidingsurface / pressuresurface / vel / pressure。タンクの3,145点をいくら入れても無視されたのは当然だった。

- ・名前が役割を決めている。同じ箱から作ったVDBを、名前だけ変えて合流させた。source なら3,588粒が湧き、surface (flipsurface の既定) や notasource では5粒のまま何も起きない。つまり flipsurface をそのままつないでも動かず、名前を source に変える必要がある。

- ・粒の数は体積を表していなかった。水位を動かして水の体積を5倍にしても、粒の数は約19,500でほとんど変わらない (一番多い19,973と一番少ない19,081の差はわずか4.7%)。体積が5倍になっているのに、である。

- ・原因は donarrowband (narrow band、既定で有効) だった。ヘルプに「水位より下の水はボリュームと場で表される」と書いてある。粒は水面付近の帯にしか置かれていないので、深さを増やしても粒は増えない。

- ・narrow band を切ると、粒の数は体積にきれいに比例した。粒の数 = $3415.7 \times \text{体積} + 18130$ で、5点すべてずれが1%未満 (最大 0.74%)。

- ・切片の 18,130 は、narrow band ありのときの粒の数 (平均 19,592) に近い。体積に比例しない分が、ちょうど帯の分に相当する。2つの測定が別々の道から同じ量を指している。

- ・体積が増えた分の粒1つが担う体積は 0.000293 で、これは粒の間隔の3乗 ($\text{sep}^3 = 0.000512$) の 0.572倍。粒は間隔の立方格子より密に詰まっている。017で「粒の数 × 間隔の3乗」では体積を表せないと書いたが、係数はこの 0.572 だった。

- ・017で立てた予想は逆だった。「reseeding を切れば粒の数が完全に保たれるはず」と書いていたが、実測では reseeding ありのほうが保たれる (40フレームで 0.20% しか動かない)。切ると36.28%も増える。

- ・理由はヘルプに書いてある。境界では水位を保つために粒が絶えず作られ、また消されている。reseeding はその「消す」側を担っているので、切ると作られた分が溜まり続ける。reseeding は粒を増やす仕組みではなく、数を一定に保つ仕組みだった。

- ・床を置かないと粒は容器の下から抜けて消える。落下する水を撮ったとき、粒の数は湧く量と消える量が釣り合って約3,400で落ち着いた。床 (useground) を入れると水が溜まり、約4,450で落ち着いた。

- ・ヘルプを読むのが一番速かった。017ではネットも配線の総当たりも試して進まなかった。付属の nodes.zip に入力の意味が書いてあり、それを読んだ時点でほぼ解けた。手元の一次情報を先に当たる。

次にやること

- ・particlefluidsurface で粒に表面を張る。粒のままだと液体に見えない
- ・衝突するもの (Collisions チャンネル) を入れて、物にぶつかる水を作る
- ・020 として POP パーティクルに進む

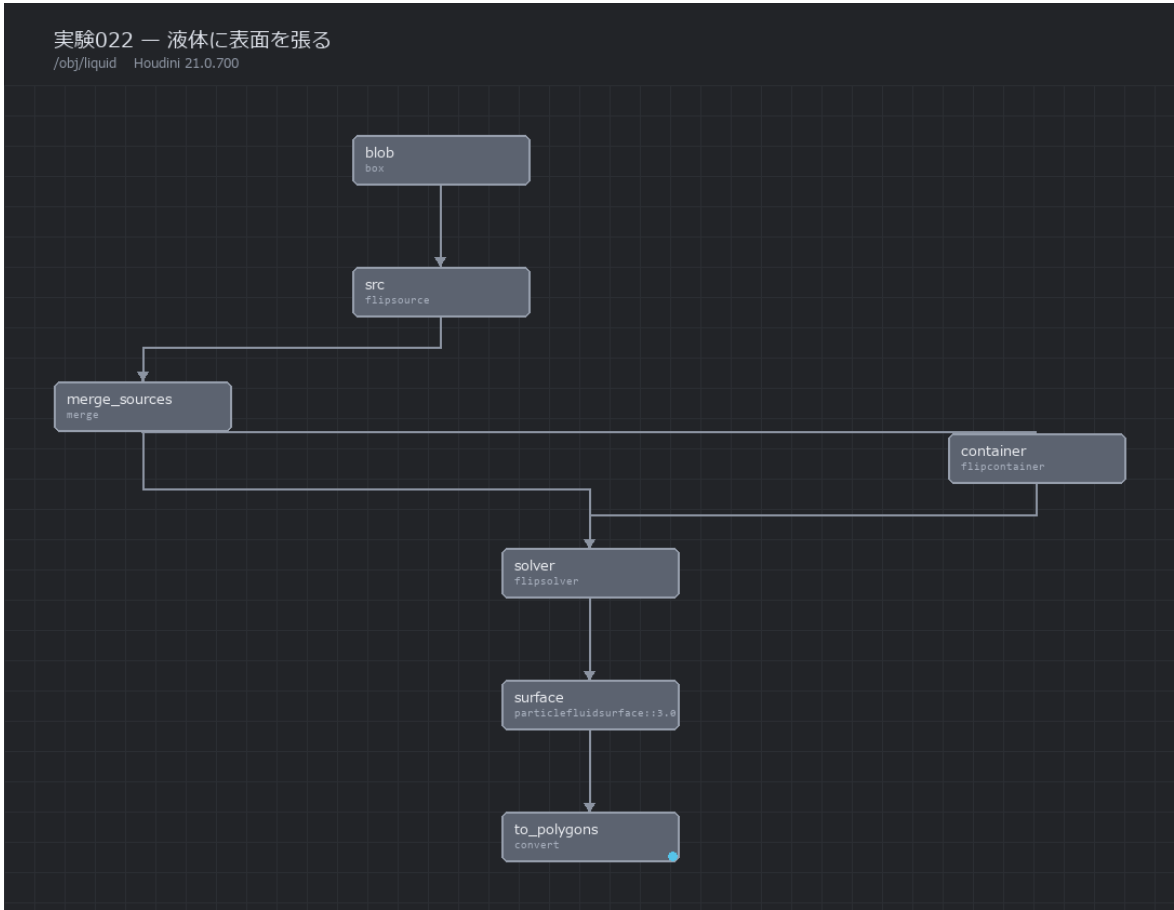
液体に表面を張る — そして019の係数が間違っていた

2026-09-24 | Houdini 21.0.700 | /obj/liquid | ノード 7 / 接続 8

何を作ったか

実験019では粒のままだったので、液体に見えなかった。particlefluidsurface で表面を張る。
張るだけなら簡単だったが、体積を測ったら019から予測した値の2.75倍になった。原因を追ったところ、2つの別々の問題が重なっていた。ひとつは境界の外側に作られる粒まで面に含めていたこと。もうひとつは019の係数そのものが間違っていたこと。
最後は、粒1つが担う体積はちょうど粒の間隔の3乗である、というきれいな形に落ち着いた。比は1.0006 だった。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
container	flipcontainer	particlesep = 0.07 size_x = 3.0 size_y = 3.0 size_z = 3.0

ノード	タイプ	デフォルトから変更したパラメータ
blob	box	size _x = 0.9 size _y = 0.9 size _z = 0.9 ty = 1.0
src	flipsource	particlesep = 0.07 volumename = source
merge_sources	merge	(変更なし)
solver	flipsolver	donarrowband = 0 folder14 = 1 folder15 = 1 folder2 = 1 folder3 = 1 folder5 = 1 folder6 = 1 ground_posy = -1.5 particlesep = 0.07 useground = 1 vis_ramp1cb = 1.0 vis_ramp2cb = 1.0 …ほか 7 件
surface	particlefluidsurface:: 3.0	closedends = 1 dobbox = 1 folder4 = 1 folder5 = 1 folder6 = 1 folder7 = 1 maskvisramp1cb = 1.0 maskvisramp1cr = 0.2 maskvisramp2cb = 1.0 maskvisramp2cg = 0.85 maskvisramp2pos = 0.25 maskvisramp3cb = 0.10000000149011612 …ほか 27 件
to_polygons	convert	(変更なし)

体積が分かっている水で確かめる

器を水位まで満たせば体積は計算で出せる。3×3 の器を深さ1.5まで満たすので 13.5。ここで面を張って 13.5 が出れば、面の張り方は正しい。

切り取り	影響半径	体積	本当の体積との比	ずれ
なし	2.0	60.0394	4.4474	+344.7%
なし	3.0	59.7756	4.4278	+342.8%
あり	2.0	13.3606	0.9897	−1.0%
あり	3.0	13.2493	0.9814	−1.9%

器の中と外で、粒はどう分かれているか

水位 0.0、粒の間隔 0.07 のとき。器は 3×3×3 で原点中心、水は y が −1.5 から 0 まで。

場所	粒の数	割合
器の中（水のある範囲）	40,871	50.4%

場所	粒の数	割合
器の外	40,235	49.6%
合計	81,106	100%

水位を変えたときの、粒の増え方

水位 -0.5（体積9.0）と

+0.5（体積18.0）で数えた。体積が9.0増えたときに、どこの粒が何個増えたか。

水位	水の体積	全部の粒	器の中	器の外
-0.50	9.0000	60,152	27,688	32,464
0.50	18.0000	101,884	53,912	47,972
増えた分	9.0000	41,732	26,224	15,508

粒1つが担う体積

体積1あたり何個増えるかの逆数。粒の間隔は 0.07 なので、その3乗は 0.000343。

数え方	体積1あたりの粒	粒1つぶんの体積	間隔の3乗との比
全部の粒	4,636.9	0.000216	0.63
器の中だけ	2,913.8	0.000343	1.0006
器の外だけ	1,723.1	—	—

わかったこと

- ・まず面が正しく張れているかを確認めた。落ちて広がった水でいきなり測ると、正解が分からないので合っているか判断できない。器を水位まで満たせば体積は計算で出せるので、そちらで先に確かめた。

- ・切り取らないと体積が4.4倍になる。器を深さ1.5まで満たした水（体積 13.5）に面を張ると、60.04 になった。付属ヘルプ (flipcontainer) に理由が書いてある。「境界の外側にも粒が作られる。面を張るときこの粒も数えられるので、メッシュが大きくなる。Bounding Box で外側を切り取れる」。

- ・切り取ると 13.36 になり、ずれは 1.0% まで下がった。面の張り方そのものは正しい。

- ・切り取るときは「切り口を閉じる」も入れる。これを入れないと、切った面が開いたままになる。開いた面には「囲まれた体積」という考え方が成り立たず、測ると 0.016 というでたらめな値が出る。最初これで一度つまずいた。

- ・境界の粒は想像よりずっと多い。水位0.0のとき、粒の 49.6% が器の外にあった。半分である。面を張るときだけでなく、粒の数を何かの目安に使うときも、この分を除かないと話が合わない。

- ・そして実験019の係数が間違っていた。019では水位を変えたときの傾きから「増えた粒1つが担う体積」を 0.000293 と出した。切片を分けたので境界の分は除けているつもりだったが、除けていなかった。

- ・理由は、境界の粒も水位と一緒に増えるから。器の側面に沿って作られるので、水が深くなるほど側面の濡れる面積が増え、粒も増える。実測では、体積が9.0増えたときに増えた 41,732個のうち 15,508個（37.2%）が器の外だった。傾きにこの分が混ざっていた。

- ・器の中の粒だけで数え直すと、体積1あたり 2,913.8個。粒1つぶんは 0.000343 になる。粒の間隔の3乗 ($0.07^3 = 0.000343$) との比は 1.0006。

- ・つまり「水の体積 = 器の中の粒の数 × 粒の間隔の3乗」が、そのまま成り立つ。019で「0.572倍」と書いた半端な係数は、境界の粒が混ざったせいで出ていた見かけの数字だった。

- ・面の太さを決めるのは influenceradius（粒の間隔の何倍まで届くか、既定3.0）。1.0まで下げると「Droplet Scale must not exceed Influence Scale」というエラーで面が作れない。2.5を超えると体積が変わらなくなる（2.5で3.49395、3.0で3.49347）。

- ・面をなめらかにすると体積は減る。0回で 3.4935、8回で 3.1243 と 10.6% 減った。実験002で「分割ではなく平滑化が形を縮ませる」と分かったが、同じことがここでも起きている。

- ・出力は

PolySoup（多数の面を1つにまとめた形）なので、そのままでは体積を測る仕組みが使えない。convertで普通のポリゴンに戻すと測れる。なお面ごとの measuredvolume を足しても体積は出ない（平らな面には体積が無い）。三角形ごとに原点との四面体の符号付き体積を足す方法で求めた。

次にやること

- ・019の記事に訂正の注記を出す。読んだ人が気づけないため
- ・023 として炎を出す。021で shredding が効かなかったのを確かめ直す
- ・024 として、動く煙で Karma を撮り直す

MPM に入る — 落ち方は式に乗る。ただし時刻が0.0742フレーム先に進んでいる

2026-09-24 | Houdini 21.0.700 | /obj/mpm | ノード 4 / 接続 4

何を作ったか

MPM (Material Point Method) は Houdini 21 で入った新しいソルバ。砂・雪・泥・パンのような「固体と流体の中間」を扱える。

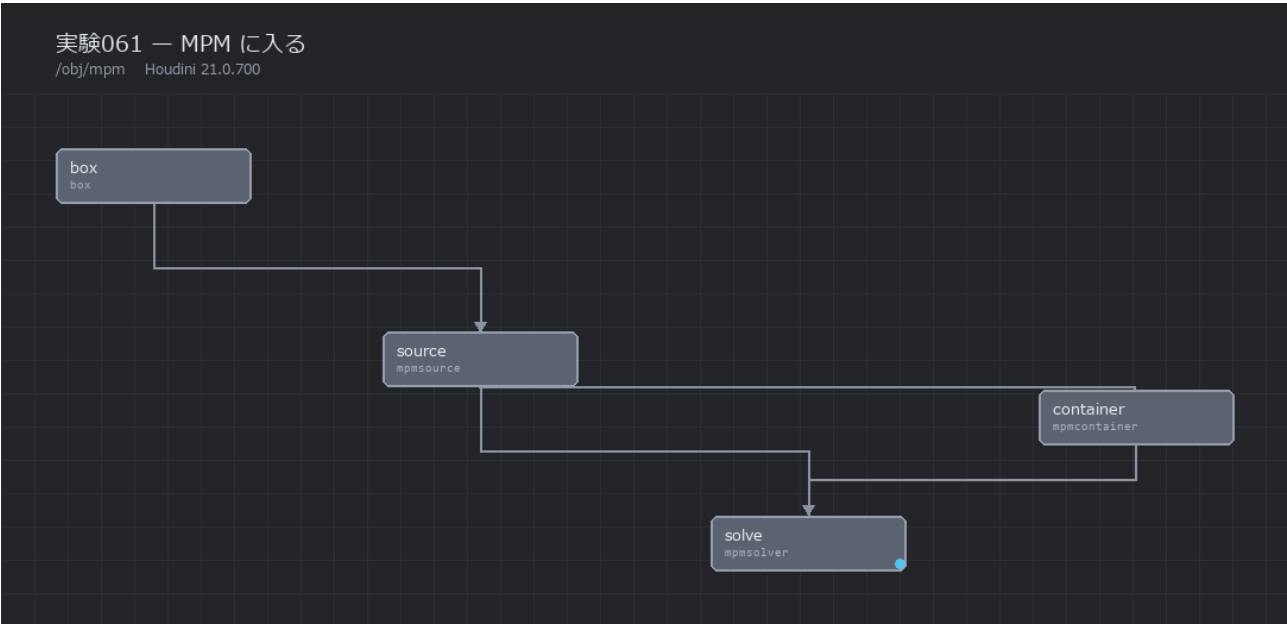
新しい分野なので、実験042 (グルーム) や実験050 (APEX) と同じ順番で入る。部品の数进行を数える・いちばん小さいものを動かす・保たれるはずの量を測る。

部品は SOP が8種、DOP が2種。いちばん小さい構成は box → mpmsource → mpmsolver の3つに、範囲を決める mpmcontainer を足した4ノードだった。1辺 1.0 の箱を粒の間隔 0.08 で埋めると 1,973個になる。

粒の数は7フレームすべてで 1,973 のまま。幅 0。横の広がりも 0.999492 から 0.999496 とほぼ動かない。自由落下しているので当然だが、数が保たれることは先に確かめておく。

落ち方は自由落下の式 ($y = y_0 - \frac{1}{2}gt^2$) に近いが、ぴったりではない。ずれは 0.000000 から -0.015201 へ単調に増える。しかもそのずれは時間に比例している。そこで「時刻が 0.003091秒 (0.0742フレーム) 。これを入れるとずれの最大は 0.000374 になり、41分の1に縮んだ。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
box	box	ty = 6.0
container	mpmcontainer	centery = 4.0 particlesep = 0.08 sizex = 6.0 sizey = 12.0 sizez = 6.0

ノード	タイプ	デフォルトから変更したパラメータ
source	mpmsource	(変更なし)
solve	mpmsolver	gravityy = -9.80665 remaptocolor1cb = 1.0 remaptocolor1cr = 0.20000000298023224 remaptocolor2cb = 1.0 remaptocolor2cg = 0.8500000238418579 remaptocolor2pos = 0.25 remaptocolor3cb = 0.10000000149011612 remaptocolor3cg = 1.0 remaptocolor3pos = 0.5 remaptocolor4cg = 1.0 remaptocolor4cr = 0.949999988079071 remaptocolor4pos = 0.75 …ほか 2 件

箱を粒にして落とす

1辺 1.0 の箱を、粒の間隔 0.08 で 1,973個の粒に変えた。重力だけ。カメラは2枚とも同じ。



フレーム1。高さ 5.998873。まだ箱の形をしている。



フレーム13。高さ 4.757841。形はほぼそのまま落ちている。

frame	秒	粒の数	高さの平均	式の高さ	ずれ	横の広がり
1	0.0000	1,973	5.998873	5.998873	0.000000	0.999492
3	0.0833	1,973	5.961985	5.964822	-0.002838	0.999492
5	0.1667	1,973	5.857198	5.862670	-0.005472	0.999493
7	0.2500	1,973	5.684511	5.692415	-0.007905	0.999493
9	0.3333	1,973	5.443722	5.454059	-0.010337	0.999494
11	0.4167	1,973	5.134832	5.147602	-0.012769	0.999495
13	0.5000	1,973	4.757841	4.773042	-0.015201	0.999496

ずれを「時刻の先走り」で説明できるか

$y = y_0 - \frac{1}{2}g(t + \delta)^2$ の δ を、いちばん遠い点（frame 13）から出して全部に当てた。 $\delta = 0.003091$ 秒 = 0.0742フレーム。

frame	式 (δなし)	式 (δ入り)	ずれ (δなし)	ずれ (δ入り)
1	5.998873	5.998826	0.000000	+0.000047
3	5.964822	5.962250	-0.002838	-0.000265
5	5.862670	5.857571	-0.005472	-0.000374
7	5.692415	5.684791	-0.007905	-0.000280
9	5.454059	5.443910	-0.010337	-0.000188
11	5.147602	5.134926	-0.012769	-0.000094
13	4.773042	4.757841	-0.015201	0.000000

粒の数と、かかる時間

1フレーム進めるのにかった時間。1回目は準備の分が乗るので捨てている。

粒の間隔	粒の数	1フレーム進める時間	粒1つあたり
0.16	238	0.068 秒	284.32 μs
0.12	583	0.059 秒	101.62 μs
0.08	1,973	0.064 秒	32.21 μs
0.06	4,602	0.093 秒	20.11 μs

わかったこと

- ・MPM の部品は SOP が8種、DOP が2種。グルームの26種や APEX の2,220種と比べると、ずいぶん小さい。新しく入ったばかりの分野で、まだ道具が少ない。
- ・いちばん小さい構成は4ノード。box → mpmsource → mpmsolver に、範囲を決める mpmcontainer を足す。mpmsolver の入力は 0=源 / 1=ぶつかるもの / 2=範囲。
- ・粒の数は完全に保たれた。7フレームすべてで 1,973個、幅 0。横の広がりも 0.999492 → 0.999496 とほぼ動かない。
- ・落ち方は自由落下の式に近いが、ぴったりではない。ずれは 0.000000 から -0.015201 へ単調に増える。「ほぼ合っている」で済ませずに、ずれの形を見る。
- ・ずれは時間に比例していた。だから「大きさが違う」のではなく「時刻がずれている」と考えられる。 $y = y_0 - \frac{1}{2}g(t + \delta)^2$ の δ を出すと 0.003091秒 (0.0742フレーム) 。
- ・ δ を入れるとずれは 41分の1 になった。0.015201 → 0.000374。ソルバは、フレーム1の時点ですでに少し時間を進めていることになる。ただし 0.0742 という数字が何から来るのかは確かめていないので、そこは断定しない。
- ・粒1つあたりの時間は、増やすほど安くなる。238個で 284.32μs、4,602個で 20.11μs。14分の1。1フレーム進める時間はどれも 0.06～0.09秒で、ほとんどが固定費だった。
- ・この入り方は3回目になる。実験042 (グルーム)、実験050 (APEX)、そして今回。部品数を数える
- ・いちばん小さいものを動かす・保たれる量を測る。新しい分野でも、順番は変えなくてよい。

次にやること

- ・ぶつかるもの (mpmcollider) を入れて、跳ね方を測る
- ・材質 (砂・雪・泥) を変えると何がかわるか

- ・ MPM の体積は保たれるか

MPM でぶつける — 砂は 16% 膨らみ、既定の塊は 31% 縮む

2026-09-24 | Houdini 21.0.700 | /obj/mpm | ノード 6 / 接続 7

何を作ったか

実験061で、MPM の粒は自由落下の式に乗ることを確かめた。今回は地面にぶつける。

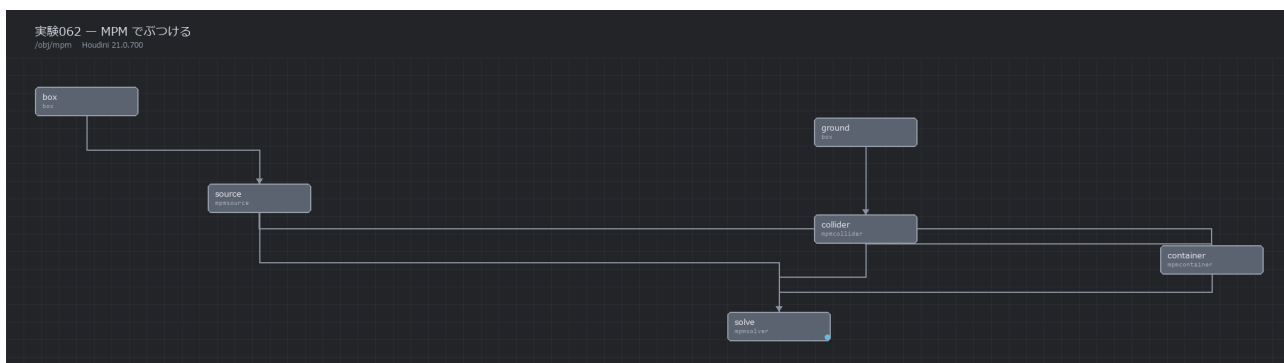
粒は Je と Jp というアトリビュートを持っている。これは体積の比で、Je が弾性（戻る）、Jp が塑性（戻らない）ぶん。掛けた $Je \times Jp$ が 1 なら体積が保たれていることになる。この量が測れるのは MPM の良いところで、実験012の boolean のように「保たれるはずの量」を直接見られる。

まず落とし穴。mpmsource には材質の見本（materialpreset）が10種類ある。Concrete / Honey / Jello / Metal / Mud / Rubber / Sand / Snow / Soil / Water。10種すべてで走らせたところ、広がりも体積比も小数点以下9桁までまったく同じ値になった。スクリプトから入れても効いていない。実際に効くのは materialtype のほうで、こちらは Elastic / Chunky / Liquid / Viscous / Sandy の5種。

その5種で測ると、まるで違う。広がりも Elastic の 1.045 から Liquid の 4.604 まで 4.4倍の開き。そして体積比が面白い。Elastic は 1.003873 とほぼ保たれ、Liquid も 0.995975。ところが既定の Chunky は 0.689523 で 31% も縮む。逆に Sandy は 1.160812 で 16% 膨らむ。

粒の数はどの材質でも 577 のまま。数は保たれるが、体積は保たれない。

ノードの接続



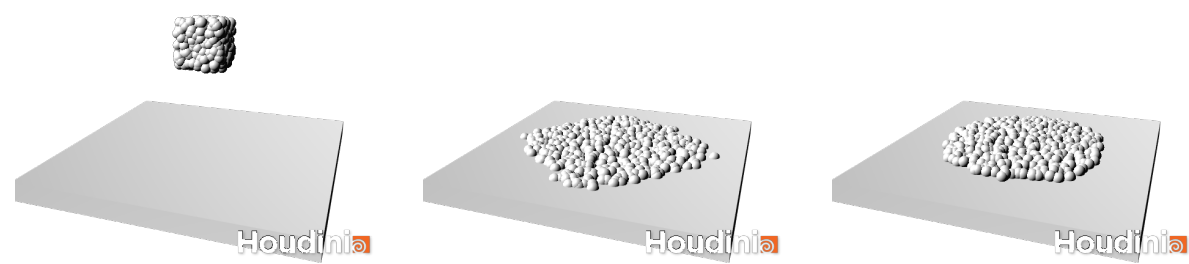
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
box	box	ty = 3.0
container	mpmcontainer	centery = 2.0 particlesep = 0.12 sizex = 8.0 sizey = 6.0 sizez = 8.0
source	mpmsource	materialpreset = 10
ground	box	sizex = 6.0 sizey = 0.4 sizez = 6.0 ty = -0.2
collider	mpmcollider	(変更なし)

ノード	タイプ	デフォルトから変更したパラメータ
solve	mpmsolver	remaptocolor1cb = 1.0 remaptocolor1cr = 0.20000000298023224 remaptocolor2cb = 1.0 remaptocolor2cg = 0.85000000238418579 remaptocolor2pos = 0.25 remaptocolor3cb = 0.10000000149011612 remaptocolor3cg = 1.0 remaptocolor3pos = 0.5 remaptocolor4cg = 1.0 remaptocolor4cr = 0.9499999988079071 remaptocolor4pos = 0.75 remaptocolor5cr = 1.0 …ほか 1 件

材質で、まるで違うものになる

1辺 1.0 の箱を高さ 3.0 から落として、地面にぶつけた。40フレーム目。粒 577個。カメラは3枚とも同じ。



Elastic。ほとんど広がらず（1.045）、体積比 1.003873。

Liquid。地面いっぱいに広がり（4.604）、体積比 0.995975。

Sandy。広がり 3.660、体積比 1.160812 で 16% 膨らむ。

materialtype	粒の数	高さの平均	いちばん上	広がり	Je	Jp	Je × Jp
Elastic（弾む）	577	2.459551	2.970170	1.04517	1.003873	1.000000	1.003873
Chunky（既定・塊）	577	0.303481	0.664286	1.09022	0.998771	0.690437	0.689523
Liquid（液体）	577	0.027283	0.125430	4.60447	0.995975	1.000000	0.995975
Viscous（粘る）	577	0.142558	0.334949	2.34138	0.972499	1.015055	0.967037
Sandy（砂）	577	0.105796	0.234545	3.65954	0.995714	1.165120	1.160812

既定（Chunky）で、ぶつかる前と後

同じシーンをフレームごとに見たもの。地面に着くのは15～20フレームのあいだ。

frame	粒の数	高さの平均	いちばん下	広がり	Je	Jp	Je × Jp
1	577	3.000636	2.500115	0.99925	1.000000	1.000000	1.000000
5	577	2.856296	2.355774	0.99925	1.000000	1.000000	1.000000

frame	粒の数	高さの平均	いちばん下	広がり	Je	Jp	Je × Jp
10	577	2.294264	1.793741	0.99925	1.000000	1.000000	1.000000
15	577	1.306452	0.805927	0.99925	1.000000	1.000000	1.000000
20	577	0.280004	0.037387	1.09691	0.992412	0.687157	0.683282
30	577	0.303183	0.056468	1.09055	0.998217	0.690437	0.689148
40	577	0.303481	0.056544	1.09022	0.998771	0.690437	0.689523

materialpreset は効かなかった（10種すべて同じ）

mpmsource の materialpreset を10種すべて試した結果。40フレーム目。

材質の見本	粒の数	高さの平均	広がり	Je × Jp
Concrete	577	0.303481	1.09022	0.689523
Honey	577	0.303481	1.09022	0.689523
Jello	577	0.303481	1.09022	0.689523
Metal	577	0.303481	1.09022	0.689523
Mud	577	0.303481	1.09022	0.689523
Rubber	577	0.303481	1.09022	0.689523
Sand	577	0.303481	1.09022	0.689523
Snow	577	0.303481	1.09022	0.689523
Soil	577	0.303481	1.09022	0.689523
Water	577	0.303481	1.09022	0.689523

わかったこと

- ・粒の数はどの材質でも 577 のまま。7フレーム × 15通り、ひとつも減らない。数は保たれる。
- ・ところが体積は保たれない。既定の Chunky で $Je \times Jp = 0.689523$ 。31% 縮んでいる。
- ・縮むのは Jp（塑性）のほう。Je は 0.998771 とほぼ 1 のまま。Jp が 0.690437 まで落ちている。ぶつかった瞬間に潰れて、そのまま戻らない。
- ・Sandy は逆に膨らむ。 $Je \times Jp = 1.160812$ 。16% 増える。Jp が 1.165120。砂は押されると粒どうしが噛み合っかさが増える——実際の砂で知られている性質が、そのまま数字に出ている。
- ・Elastic と Liquid はほぼ保たれる。1.003873 と 0.995975。どちらも 0.4% 以内。Jp が 1.000000 のまま動かないので、潰れても戻る。
- ・広がり方は 4.4倍の開きがある。Elastic 1.045、Liquid 4.604。同じ形を同じ高さから落としているのに、地面に着いたあとがまったく違う。
- ・Elastic だけは落ちきらない。高さの平均が 2.459551 で、いちばん上は 2.970170。跳ね返って空中にいる。ほかの4種は 0.03~0.30 まで落ちている。
- ・そして落とし穴。materialpreset はスクリプトから入れても効かない。10種すべてで、広がりも体積比も小数点以下9桁までまったく同じ。エラーも警告も出ない。
- ・実際に効くのは materialtype。Elastic / Chunky / Liquid / Viscous / Sandy の5種で、既定は Chunky。見本のメニューは、押したときに値を流し込むための仕掛けで、値そのものではない。

- ・この形の罫は4度目になる。実験044のガイド、実験041の bump、実験050の可変長の入力、そして今回。「入れたのに効かない。しかも何も言われない」。見分け方はいつも同じで、値を大きく変えて出力が動くかを見る。

次にやること

- ・ materialtype ごとのパラメータ（粘り・硬さ）を1つずつ動かす
- ・ mpmsurface で粒を面にする
- ・ MPM の時間の刻み（substep）と結果の関係

MPM の粒を面に戻す — 27% 太る。膨らむ厚みは粒の間隔の 0.4倍

2026-09-24 | Houdini 21.0.700 | /obj/mpm | ノード 5 / 接続 4

何を作ったか

実験061・062で、MPM

は物を粒の集まりとして持つと分かった。ただし粒のままではレンダできない。mpmsurface
で面に戻す。

ここで確かめられることがある。体積 1.0 の箱を粒にして、面に戻したら、また 1.0
になるか。実験059で obj

の往復を測ったのと同じ考え方で、今度は形式ではなく「粒を経由すること」で何が変わるかを見る。

結果は 1.2663。27% 太る。理由は絵を見れば分かる。面は粒の一つひとつを外側から包むので、元の箱
より一回り大きくなる。

その厚みも出せる。1辺が $1 + 2d$ に太ったと考えて逆算すると、粒の間隔 0.16 で $d = 0.05774$ 、0.08
で $d = 0.03538$ 。間隔で割ると $0.3609 / 0.4261 / 0.4094 / 0.4423$ 。つまり膨らむ厚みは粒の間隔の
0.4倍くらいで、間隔を細かくすれば太り方も減る。

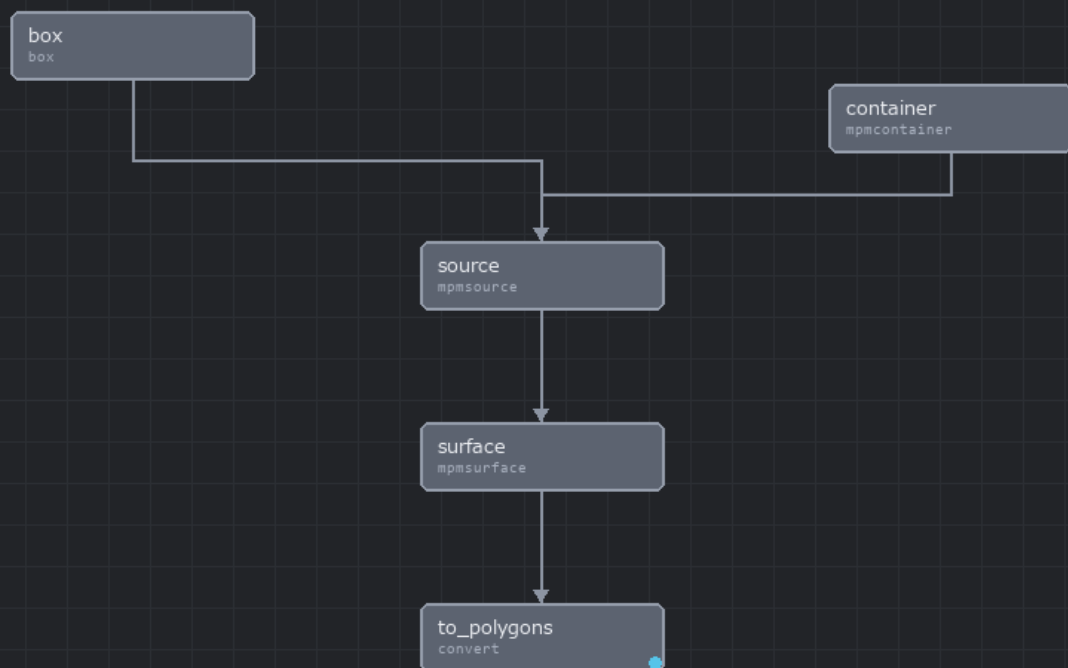
面の細かさ (voxelscale) はほとんど効かない。1.0 / 0.75 / 0.5 で体積は 1.2663 / 1.2705 / 1.2736
と 0.6% しか変わらないのに、点の数は 5,495 から 23,485 へ 4.3倍になる。しかも 1.5 と 2.0
では何も出てこない。

もうひとつ。面の作り方には「VDB from Particles」のほかに「Neural Point
Surface」という機械学習の方法がある。こちらは点が 2,168 と 2.5分の1なのに体積は 1.1882
で元に近い。ただし 1.35秒かかり、3倍近く遅い。

ノードの接続

実験063 — MPM の粒を面に戻す

/obj/mpm Houdini 21.0.700

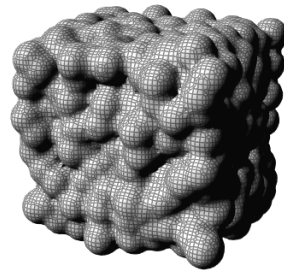


スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
box	box	(変更なし)
container	mpmcontainer	particlesep = 0.1 sizex = 4.0 sizey = 4.0 sizez = 4.0
source	mpmsource	(変更なし)
surface	mpmsurface	outputtype = 2 voxelscale = 0.75
to_polygons	convert	(変更なし)

粒を包むから太る

1辺 1.0 の箱を、粒の間隔 0.1 で 1,001個の粒にしてから面に戻した。



Houdini

Houdini

voxelscale 2.0。粗すぎて何も出てこない（点 0 / 面 0）。

voxelscale 0.5。23,485点。粒ひとつずつの丸みがそのまま面になる。

voxelscale	点	面	体積	元との比	秒
2.0	0	0	0.000000	—	0.21
1.5	0	0	0.000000	—	0.21
1.0	5,495	5,488	1.266304	1.2663	0.22
0.75	10,180	10,176	1.270461	1.2705	0.46
0.5	23,485	23,493	1.273598	1.2736	0.45

粒の間隔を変えると、太り方はどう変わるか

voxelscale は既定のまま。1辺が $1 + 2d$ に太ったと考えて d を逆算した。

粒の間隔	粒	点	面	体積	元との比	$1 + 2d$	d	$d \div \text{間隔}$
0.16	231	2,278	2,272	1.387964	1.3880	1.11547	0.05774	0.3609
0.12	584	3,957	3,950	1.339217	1.3392	1.10226	0.05113	0.4261
0.10	1,001	5,495	5,488	1.266304	1.2663	1.08188	0.04094	0.4094
0.08	1,956	8,589	8,574	1.227668	1.2277	1.07076	0.03538	0.4423

面の作り方を变える

同じ粒（1,001個）から、2つの方法で面を作った。

やり方	点	面	体積	元との比	秒
VDB from Particles	5,495	5,488	1.266304	1.2663	0.47
Neural Point Surface	2,168	2,356	1.188212	1.1882	1.35

わかったこと

- ・粒を経由すると 27% 太る。体積 1.0 の箱が 1.266304 になる。面は粒の一つひとつを外側から包むので、元より一回り大きくなる。

- ・膨らむ厚みは、粒の間隔の 0.4 倍くらいだった。1 辺が $1 + 2d$ に太ったと考えて逆算すると、 $d \div$ 間隔が $0.3609 / 0.4261 / 0.4094 / 0.4423$ 。4 通りで 0.36~0.44 に収まる。

- ・だから細かくすれば太り方は減る。間隔 0.16 で 1.3880、0.08 で 1.2277。間隔を半分にすると、太る割合もおおよそ半分になる。

- ・面の細かさ (voxelscale) はほとんど効かない。1.0 / 0.75 / 0.5 で体積が 1.2663 / 1.2705 / 1.2736。0.6% しか変わらない。

- ・それなのに点の数は 4.3 倍になる。5,495 → 23,485。重くなるだけで、形はほとんど変わらない。太り方を減らしたいなら、voxelscale ではなく粒の間隔を触る。

- ・粗すぎると何も出てこない。voxelscale 1.5 と 2.0 では点も面も 0。エラーは出ない。「0 個」も結果として記録しておく。

- ・mpmsurface が出すのは polysoup。1 プリミティブの中に全部の面が入っている。そのままだと面ごとの計算ができないので、convert でふつうのポリゴンに割ってから測った。最初これに気づかず、体積が 0.06 という妙な値になった。

- ・機械学習の方法もある。「Neural Point Surface」。点が 2,168 と 2.5 分の 1 なのに、体積は 1.1882 で元に近い。軽くて正確。

- ・ただし 3 倍近く遅い。0.47 秒 対 1.35 秒。1 フレームならどうでもいい差だが、200 フレームなら 3 分 対 4.5 分になる。

- ・出力の既定は面ではない。outputtype の既定は「Surface VDB」で、面が欲しいなら polygonmesh に変える。これも最初に掛かった。

次にやること

- ・Neural Point Surface の精度を、もっと複雑な形で測る
- ・MPM の時間の刻み (substep) と結果の関係
- ・粒の間隔と、シミュレーションそのものの結果の関係

MPM の刻みを細かくする ー 量によって、落ち着く速さが違った

2026-09-24

何を作ったか

シミュレーションは1フレームを何回かに分けて解く。その回数が substep。細かくするほど正しくなるはずだが、そのぶん時間がかかる。では、どこまで細かくすれば「落ち着いた」と言えるのか。

実験018で、煙のサンプル数とノイズの関係を測ったときと同じ問いになる。あのときは「4倍にすればノイズは半分」という式を立てて、当たったのは狭い範囲だけだった。

今回も、単純な式には乗らなかった。ただし別のことが分かった。量によって落ち着く速さがまったく違う。

高さの平均は、substep 1 でもう落ち着いている。16 との差は 0.001584 で、いちばん大きいずれでも 0.004984。一方、広がり は 1 で 1.09055、16 で 1.04412 と 4.4% 違う。8 まで上げてようやく 16 との差が 0.002012 になる。体積比も同じで、8 以上でないと落ち着かない。

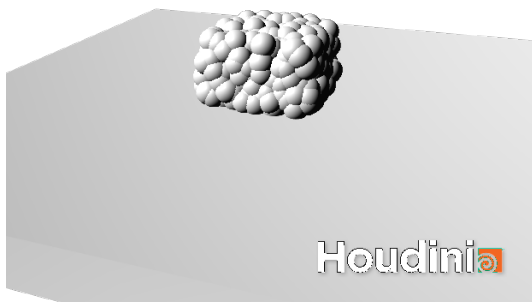
時間は substep 16 で 15.85秒。substep 1 の 0.62秒に対して 25.5倍。16倍のはずが 25.5倍なのは、substep 1 だけ特別に安い（1刻みあたり 0.621秒に対し、ほかは 0.9~1.05秒）。

そして絵で見ると、substep 1 と 16

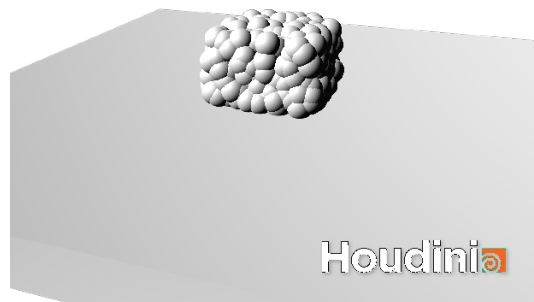
はほとんど区別がつかない。25.5倍の時間をかけて変わるのは、目より先に数字のほう。

25.5倍の時間をかけても、絵はほとんど変わらない

1辺 1.0 の箱を高さ 3.0 から落として地面にぶつけた30フレーム目。粒 577個。カメラは2枚とも同じ。



substep 1. 0.62秒。広がり 1.09055。



substep 16. 15.85秒。広がり 1.04412。

substep	粒	高さの平均	いちばん上	広がり	Je × Jp	かかった時間	1刻みあたり
1	577	0.303183	0.663563	1.09055	0.689148	0.62 秒	0.621 秒
2	577	0.304263	0.664534	1.09238	0.689409	1.85 秒	0.927 秒
4	577	0.303593	0.665196	1.07192	0.673849	3.61 秒	0.901 秒
8	577	0.299784	0.662280	1.04211	0.649739	8.42 秒	1.052 秒
16	577	0.304767	0.669925	1.04412	0.653366	15.85 秒	0.990 秒

いちばん細かい 16 を正として、どれだけずれるか

substep 16 の結果を基準にした差。

substep	高さのずれ	広がり of ずれ	Je × Jp のずれ	時間 ÷ 16 の時間
1	0.001584	0.046433	0.035782	0.039
2	0.000504	0.048263	0.036043	0.117
4	0.001174	0.027802	0.020483	0.228
8	0.004984	0.002012	0.003627	0.531
16	0.000000	0.000000	0.000000	1.000

刻みを2倍にすると、ずれは半分になるか

前の段のずれを、次の段のずれで割ったもの。2 なら「2倍にすれば半分」。

比べる組	高さのずれの比	広がり of ずれの比
1 → 2	3.141	0.962
2 → 4	0.430	1.736
4 → 8	0.236	13.817

わかったこと

- ・量によって、落ち着く速さがまったく違う。高さの平均は substep 1 でもう落ち着いている（16 との差 0.001584）。広がり と 体積比は 8 まで上げないと落ち着かない。
- ・高さは substep では変わらない。5通りすべてで 0.2998～0.3048。いちばん大きいずれでも 0.004984。重力で落ちる距離は、刻みを細かくしても変わらない。
- ・広がり は 4.4% 違う。substep 1 で 1.09055、16 で 1.04412。粗いほど広がる。ぶつかった瞬間の扱いが粗いぶん、余計に跳ねている。
- ・体積比も同じ向きに動く。0.689148（1）→ 0.653366（16）。粗いほど縮みが小さく見える。8 と 16 の差は 0.003627 まで縮む。
- ・「2倍にすればずれは半分」には乗らない。高さのずれの比が 3.141 / 0.430 / 0.236、広がりが 0.962 / 1.736 / 13.817。ばらばらで、式にならない。実験018の煙のときと同じで、単純な法則を期待すると外れる。
- ・時間は 16倍ではなく 25.5倍になる。0.62秒 → 15.85秒。substep 1 だけが特別に安い（1刻みあたり 0.621秒、ほかは 0.9～1.05秒）ため。substep 2 以上なら、時間はほぼ刻みの数に比例する。
- ・絵ではほとんど区別がつかない。25.5倍の時間をかけて変わるのは、目より先に数字のほう。見せる絵を作るだけなら、刻みを上げる前に別のところを疑ったほうがいい。
- ・実用の落としどころは 8。16 との差が 高さ 0.004984 / 広がり 0.002012 / 体積比 0.003627 まで縮んで、時間は 16 の 53%。半分の時間で、ほぼ同じ結果になる。
- ・測り方について。いちばん細かいものを「正」と決めて、そこからのずれを見ている。本当の正解は分からないので、これは「落ち着いたかどうか」を見る方法であって「正しいかどうか」ではない。16 と 32 の差も測れば、もっと確かになる。

次にやること

- ・ 粒の間隔と、シミュレーションそのものの結果の関係
- ・ MPM に風や外力を加える
- ・ MPM の粒を色分けして、どこが塑性変形したかを見る

MPM の粒を細かくする — 刻みより粒のほうが効くし、しかも安い

2026-09-24

何を作ったか

実験064では「1フレームを何回に分けるか」(substep)を変えた。今回は「物を何粒で表すか」(粒の間隔)を変える。どちらも「細かくすれば正しくなる」はずの数字だが、効き方は同じだろうか。

まったく違った。

高さの平均は、substep ではほとんど動かなかった(5通りで 0.2998~0.3048)。ところが粒を細かくすると 0.357377 から 0.275207 へ、23% も動く。しかも一本調子に下がっていく。体積比も 0.733703 から 0.649174 へ 12% 動く。

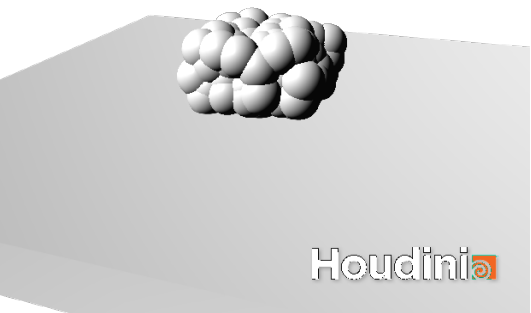
時間の効率も違う。粒を 118個から 2,916個へ 24.7倍に増やしても、時間は 3.73倍にしかない。粒1つあたりの時間は 5.411ms から 0.817ms へ 6.6分の1 に下がる。substep のほうは 16倍で 25.5倍の時間がかかったので、対照的である。

つまり、迷ったら粒を細かくするほうが得。同じ時間を払うなら、substep より粒に使ったほうが結果は動く。

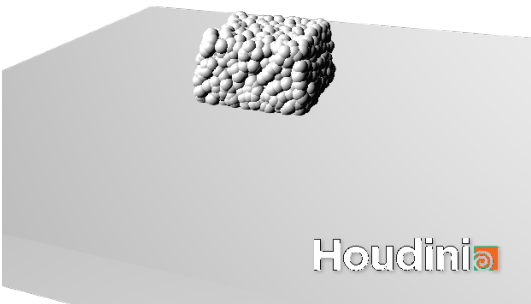
ただし広がりだけは素直でない。1.08755 / 1.07179 / 1.09055 / 1.09659 / 1.09498 / 1.07350 と上下して、どこへ落ち着くのか見えない。substep のときは 8 以上できれいに落ち着いたので、こちらも逆になっている。

粒の間隔を変える

1辺 1.0 の箱を高さ 3.0 から落として地面にぶつけた30フレーム目。substep は既定のまま。カメラは2枚とも同じ。



間隔 0.20。118粒、0.64秒。高さの平均 0.357377。



間隔 0.07。2,916粒、2.38秒。高さの平均 0.275207。

間隔	粒	高さの平均	いちばん上	広がり	Je × Jp	かかった時間	粒1つあたり
0.20	118	0.357377	0.679947	1.08755	0.733703	0.64 秒	5.411 ms
0.16	250	0.332527	0.699142	1.07179	0.712208	0.96 秒	3.858 ms
0.12	577	0.303183	0.663563	1.09055	0.689148	1.09 秒	1.896 ms

間隔	粒	高さの平均	いちばん上	広がり	Je × Jp	かかった時間	粒1つあたり
0.10	985	0.290995	0.648743	1.09659	0.681013	1.17 秒	1.192 ms
0.08	1,919	0.279317	0.636713	1.09498	0.658524	1.64 秒	0.854 ms
0.07	2,916	0.275207	0.629527	1.07350	0.649174	2.38 秒	0.817 ms

いちばん細かい 0.07 を正として、どれだけずれるか

substep のときと同じやり方で比べた。

間隔	高さのずれ	広がり	Je × Jp のずれ	時間 ÷ 0.07の時間
0.20	0.082171	0.014051	0.084529	0.268
0.16	0.057320	0.001716	0.063033	0.405
0.12	0.027977	0.017053	0.039974	0.459
0.10	0.015788	0.023091	0.031839	0.493
0.08	0.004110	0.021480	0.009350	0.687
0.07	0.000000	0.000000	0.000000	1.000

粒の数と時間の関係

いちばん粗い 0.20 を 1 としたときの比。

間隔	粒	粒の比	時間の比	時間の比 ÷ 粒の比
0.20	118	1.00	1.00	1.000
0.16	250	2.12	1.51	0.713
0.12	577	4.89	1.71	0.350
0.10	985	8.35	1.84	0.220
0.08	1,919	16.26	2.57	0.158
0.07	2,916	24.71	3.73	0.151

わかったこと

- ・高さは、粒を細かくすると 23% 動く。0.357377 → 0.275207。しかも一本調子に下がる。substep では 5通りで 0.2998～0.3048 とほとんど動かなかったの、まったく逆の結果になった。
- ・体積比も 12% 動く。0.733703 → 0.649174。粗い粒ほど「縮んでいない」ように見える。
- ・時間の効率がまるで違う。粒を 24.7倍に増やしても、時間は 3.73倍にしかない。substep は 16倍で 25.5倍だった。
- ・粒1つあたりの時間は 6.6分の1 に下がる。5.411ms → 0.817ms。粗いほど、1粒に払う固定費の割合が大きい。
- ・だから、迷ったら粒を細かくするほうが得。同じ時間を払うなら、substep より粒に使ったほうが結果は動く。substep を 16 にすると 25.5倍の時間で高さは動かないが、粒を 24.7倍にすると 3.73倍の時間で高さが 23% 動く。
- ・ただし広がりだけは素直でない。1.08755 / 1.07179 / 1.09055 / 1.09659 / 1.09498 / 1.07350 と上下して、どこへ落ち着くのか見えない。substep のときは 8 以上できれいに落ち着いたの、こちらも逆になっている。

- ・つまり「細かくすれば正しくなる」は、量ごとに別々に確かめるしかない。高さは粒で決まり、広がり
は刻みで決まる。ひとつの数字を見て「収束した」と言うと、別の数字で外れる。

- ・絵で見ると、粗いほうは粒が大きく見えるだけではない。塊としての形が違う。118粒では「箱が潰れた」
という形にならず、粒がまばらに散っているようにしか見えない。

- ・測り方について。実験064と同じ形の実験を、軸だけ変えて走らせた。同じ物差しで測ってあるから、2
つを直接比べられる。比べたいなら、測り方をそろえてから測る。

次にやること

- ・粒の間隔と substep を同時に動かして、どちらが先に効くか
- ・MPM に風や外力を加える
- ・どこが塑性変形したかを色分けする

MPM の風 — 風速だけ上げてても何も起きない。抵抗は速さの2乗だった

2026-09-24

何を作ったか

mpmsolver の Forces には Gravity のと並んで Wind Velocity と Air Drag が並んでいる。どちらも既定は 0。風速だけ 0 から 10 まで上げてみた。

何も起きなかった。位置も速度も、9桁まで完全に同じ。風は空気抵抗を通してしか効かないので、Air Drag が 0 のあいだは Wind Velocity

をいくら上げてても無意味である。実験044のガイド、実験050の可変長入力、実験062の materialpreset に続いて、これで4件目の「黙って無視されるパラメータ」になった。

そこで Air Drag を 0.25 から 4.0 まで変えて、落下速度を測った。よくある「速さに比例する抵抗」の式 $v = -g/d (1 - e^{(-dt)})$ に当てはめると、 $d = 1.0$ のとき式は -6.047555 というのに実測は -3.157398。1.92倍も外れる。

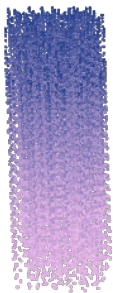
当たったのは速さの2乗に比例する抵抗のほうだった。終端速度 $\sqrt{g/d}$ は $d = 4.0$ で 1.566046、実測 1.607434。さらに横風を足して3次元の相対速度で解き直すと、横速度は実測 1.840484 / 4.728911 / 9.656235 に対して式が 1.841579 / 4.729887 / 9.656630。ずれは 0.001095 / 0.000976 / 0.000395 で、いちばん大きいところでも 0.06% である。

もうひとつ分かったことがある。mpmsolver は既定で $y = 0$ に地面を持っている（Ground Plane の groundactive が 1）。コライダーを一つも置かなくても、粒は $y = 0$ で止まる。今回は自由落下を測りたかったので、これを切った。

通った跡を1枚に重ねる

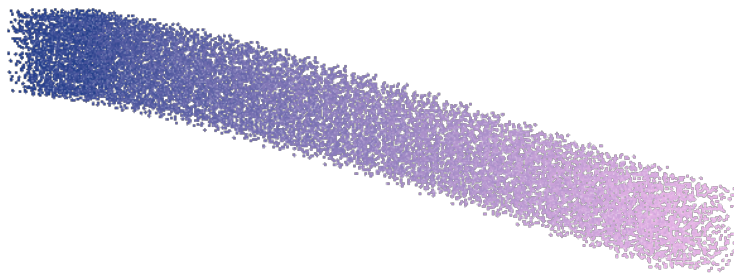
1辺 1.0 の箱を $y=3.0$

から落として、24フレーム分の粒をぜんぶ置いた。古いほど青く、新しいほど明るい。2枚とも空気抵抗 1.0、カメラも枠も同じ。



Houdini

風速 0。まっすぐ落ちる。24フレーム目の x の平均は 0.007215。



Houdini

風速 10。横へ 7.548967 まで流される。落ちる速さも -3.157398 から -3.099901 へ変わる。

A. 空気抵抗 0 のまま、風速だけ上げる

24フレーム目。地面は切ってあるので、ぶつかっていない。

風速	x の平均	vx の平均	y の平均	vy の平均	風速 0 との差
0	0.007215	0.000000	-1.542875	-9.401245	0.000000000
2	0.007215	0.000000	-1.542875	-9.401245	0.000000000
5	0.007215	0.000000	-1.542875	-9.401245	0.000000000
10	0.007215	0.000000	-1.542875	-9.401245	0.000000000

B. 風なしで空気抵抗を変える。どの式に乗るか

経過時間 $t = 23 / 24 = 0.958333$ 秒。抵抗なしなら $g t = 9.401250$ で落ちているはずで、実測は 9.401245 だった（ずれ 0.000005）。

空気抵抗 d	vy 実測	速さに比例する式	2乗に比例する式	終端速度 $\sqrt{(g/d)}$	実測 - 2乗の式
0.25	-5.704545	-8.359859	-5.670913	-6.264184	-0.033632
0.5	-4.343024	-7.469349	-4.304229	-4.429447	-0.038795
1.0	-3.157398	-6.047555	-3.116652	-3.132092	-0.040746
2.0	-2.255033	-4.183492	-2.213813	-2.214723	-0.041220
4.0	-1.607434	-2.399434	-1.566027	-1.566046	-0.041407

C. 空気抵抗 1.0 で風速を変える。横速度は風速に比例するか

「速さに比例する抵抗」なら、 $vx \div \text{風速}$ はどの風速でも同じ 0.616468 になるはずだった。

風速 w	vx 実測	$vx \div w$	2乗の式	実測 - 式	vy 実測	vy の式	x の平均
2	1.840484	0.920242	1.841579	-0.001095	-3.146102	-3.105449	1.147626
5	4.728911	0.945782	4.729887	-0.000976	-3.125549	-3.084892	3.359090
10	9.656235	0.965623	9.656630	-0.000395	-3.099901	-3.065852	7.548967

終端速度は $\sqrt{(g/d)}$ 。d を半分にすると $\sqrt{2} = 1.41421$ 倍になるはず

抵抗が強いほど早く終端に達するので、上の2組だけが式に近い。

比べる組	実測の比	$\sqrt{2}$ との差
4.0 → 2.0	1.40287	0.01134
2.0 → 1.0	1.40016	0.01405
1.0 → 0.5	1.37549	0.03872
0.5 → 0.25	1.31349	0.10072

わかったこと

- ・ Wind Velocity だけを上げてても、何も起きない。風速 0 / 2 / 5 / 10 の4通りで、位置も速度も9桁まで完全に同じ。風は空気抵抗を通してしか効かない。Air Drag が既定の 0 のままなら、風速はただの飾りである。
- ・ これで「黙って無視されるパラメータ」は4件目。実験044のガイド、実験050の可変長入力、実験062の materialpreset、そして今回の風速。見つけ方はいつも同じで、入力を極端に振って出力が動くかどうかを見る。
- ・ Houdini の空気抵抗は「速さの2乗」に比例する。よくある「速さに比例」の式は $d = 1.0$ で -6.047555 と言うが、実測は -3.157398 。1.92倍も外れる。2乗の式なら -3.116652 で、ずれは 0.040746 に縮む。
- ・ 実験029で測った POP の popwind も二乗抵抗だった。Houdini の「空気抵抗」は、ソルバが違っても同じ形をしているらしい。ただしあちらは1次元の式でずれ 0.0000003 まで合ったのに対し、今回は縦方向に 0.0414 の食い違いが残る。
- ・ 終端速度は $\sqrt{(g/d)}$ 。d = 4.0 で式が 1.566046 、実測 1.607434 。d を半分にすると速度は $\sqrt{2}$ 倍になるはずで、実測の比は $1.40287 / 1.40016$ ($\sqrt{2} = 1.41421$)。ただし抵抗が弱いと 0.958 秒ではまだ終端に届かないので、 $1.37549 / 1.31349$ と離れていく。
- ・ 抵抗は3次元の相対速度の大きさで決まる。だから横風は落ち方まで変える。風速 0 / 2 / 5 / 10 に対して v_y は $-3.157398 / -3.146102 / -3.125549 / -3.099901$ 。横に強く吹くほど、下向きの抵抗は弱くなる。
- ・ 横方向は式と 0.06% しか違わない。 v_x の実測 $1.840484 / 4.728911 / 9.656235$ に対し式は $1.841579 / 4.729887 / 9.656630$ 。ずれ $0.001095 / 0.000976 / 0.000395$ 。抵抗の式が当たっている証拠として、これがいちばん強い。
- ・ 縦方向だけ、いつも 0.0414 ほど速い。d = 0.25 → 4.0 でずれは $-0.033632 / -0.038795 / -0.040746 / -0.041220 / -0.041407$ と、抵抗の強さによらずほぼ同じ値に落ち着く。重力で割ると 0.004221 秒 (0.1013 フレーム)。抵抗を切ると速度のずれは 0.000005 まで消えるので、これは重力の刻み方ではなく抵抗の当て方から出ている。
- ・ mpsolver は既定で $y = 0$ に地面を持っている。groundactive が 1。コライダを一つも置かなくても、粒は $y = 0$ で止まる。最初この実験は地面なしのつもりで組んだのに、粒が $y \approx 0.3$ で止まって混乱した。
- ・ 実験062～065 の結果には影響しない。あちらで置いた地面の箱は size (6, 0.4, 6) を t (0, -0.2, 0) に置いたもので、上面がちょうど $y = 0$ 。組み込みの地面と同じ高さに重なっていた。
- ・ 測り方について。「速さに比例」と「2乗に比例」は、1本の曲線を見ているだけでは見分けが付きにくい。どちらもそれらしい形をしている。見分けられたのは、風速を3通りに振って $v_x \div \text{風速}$

が一定かどうかを見たから。比例なら一定になるはずが、0.920242 / 0.945782 / 0.965623 と動いた。入力を複数振って、比が保たれるかを見る。

次にやること

- ・組み込みの地面の摩擦と粘着（groundfriction / groundsticky）を測る
- ・mpmsolver の表面張力（Surface Tension）を測る
- ・粒の間隔と substep を同時に動かして、どちらが先に効くか

MPM の地面の摩擦 — 教科書の式に 0.04% で乗る。ただし塊が潰れるまで

2026-09-24

何を作ったか

実験066で、mpmsolver が既定で $y = 0$ に地面を持っていると分かった。その地面には Friction（既定 1.0）と Sticky（既定 0.0）が付いている。1辺 1.0 の塊を横向きに初速 4.0 で投げて、摩擦を 0.0 から 2.0 まで変えた。

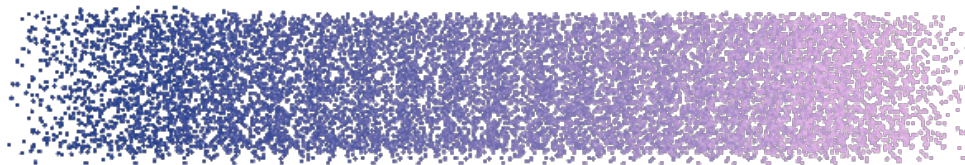
高校物理の式は「止まるまでの距離 = 初速² ÷ (2 × 摩擦係数 × 重力)」と言う。摩擦 0.25 のとき式は 3.261978、実測は 3.260808。ずれは 0.04% しかない。減速そのものも測った。摩擦 0.1 のとき 1.625秒で速さが 1.592067 減っており、1秒あたり 0.979733。 $\mu \times g = 0.981$ に対して 0.13% の差である。初速を 2.0 / 3.0 / 4.0 と変えると、距離 ÷ 初速² は 0.202727 / 0.203753 / 0.203801 とほぼ一定になる。つまり Houdini の Ground Friction は、そのままクーロン摩擦の係数だった。

ところが摩擦を上げていくと式から外れ始める。0.5 で 7.4%、1.0 で 46.5%、2.0 では 2.8倍も遠くまで進む。原因は塊が潰れることにある。摩擦 0.0 では横の広がり 0.995852 のまま形を保つのに、2.0 では 2.374637 まで広がり、高さは 0.502654 から 0.325032 へ下がる。止まるのではなく、つぶれて広がっている。だから摩擦を 1.0 から 2.0 へ倍にしても、距離は 1.194665 から 1.148402 へ 4% しか縮まない。

Sticky のほうは、横滑りにはほとんど効かない。0.25 / 0.5 / 1.0 で距離は 3.240815 / 3.240539 / 3.240539 で、0.5 と 1.0 は6桁まで同じ。ただし落として跳ねさせると違いが出る。跳ね返りの高さが 0.026922 から 0.013617 へ、ちょうど半分になる。Sticky は「離れるとき」に効く数字であって、「滑るとき」の数字ではない。

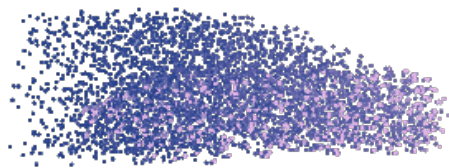
通った跡を1枚に重ねる

1辺 1.0 の塊を $y = 0.52$ から横向きに初速 4.0 で投げた。40フレーム分を2フレームおきに重ねている。古いほど青く、新しいほど明るい。2枚ともカメラも枠も同じ。



Houdini

摩擦 0.1。5.204091 進んでも、まだ 2.407933 の速さで動いている。形はほぼ保たれている（横の広がり 1.011225）。



Houdini

摩擦 2.0。1.148402 で止まる。ただし止まったというより潰れている（横の広がり 2.374637、高さ 0.325032）。

A. 摩擦を変えて、横に投げた塊がどこまで滑るか

40フレーム目。経過時間 1.625 秒。摩擦 0.0 の距離 6.499988 は、初速 4.0×1.625 秒 = 6.5 とのずれが 0.000012。

摩擦	滑った距離	最後の vx	高さ	横の広がり	止まった秒
0.0	6.499988	3.999986	0.502654	0.995852	止まらず
0.1	5.204091	2.407933	0.501820	1.011225	止まらず
0.25	3.260808	0.025688	0.504465	1.049312	40フレームでほぼ 0
0.5	1.751774	-0.008058	0.355074	1.929583	1.0000
1.0	1.194665	0.000409	0.340183	2.138179	0.8750
2.0	1.148402	-0.006202	0.325032	2.374637	0.8333

B. 教科書の式「止まるまでの距離 = 初速² ÷ (2 μ g)」に乗るか

摩擦 0.1 はまだ止まっていないので、比べられない（式が言う止まるまでの時間は 4.077 秒で、測った 1.625 秒より長い）。

摩擦 μ	実測の距離	式の距離	実測 ÷ 式	式が言う止まる時間
0.0	6.499988	∞	—	∞
0.1	5.204091	8.154944	まだ止まっていない	4.077472
0.25	3.260808	3.261978	0.999642	1.630989
0.5	1.751774	1.630989	1.074056	0.815494
1.0	1.194665	0.815494	1.464958	0.407747
2.0	1.148402	0.407747	2.816457	0.203874

B2. 減速そのものを測る。μ × g と合うか

まだ止まっていないものだけを対象にした。止まったあとの時間が混ざると平均が薄まるため。

摩擦 μ	vx の減り	÷ 経過秒 1.625	μ × g	実測 ÷ μg
0.0	0.000014	0.000009	0.000000	—

摩擦 μ	v_x の減り	$\div$ 経過秒 1.625	$\mu \times g$	実測 $\div \mu g$
0.1	1.592067	0.979733	0.981000	0.998709

C. 粘着を変える（摩擦は 0.25 に固定）

0.5 と 1.0 は6桁まで完全に同じ値。

粘着	滑った距離	最後の v_x	高さ	横の広がり
0.0	3.260808	0.025688	0.504465	1.049312
0.25	3.240815	0.010982	0.503325	1.045604
0.5	3.240539	0.010746	0.503321	1.045606
1.0	3.240539	0.010746	0.503321	1.045606

D. 初速を変える（摩擦 0.25・粘着 0）。距離は初速の2乗に比例するか

比例していれば「距離 $\div$ 初速²」が一定になる。

初速	滑った距離	距離 $\div$ 初速 ²	式の距離	実測 $\div$ 式
2.0	0.810910	0.202727	0.815494	0.994378
3.0	1.833778	0.203753	1.834862	0.999409
4.0	3.260808	0.203801	3.261978	0.999642

E. 落として跳ねさせる（摩擦 0.25）。粘着は跳ね返りを止めるか

初速 0 で $y = 2.5$ から落とした。いちばん低くなるのは4通りとも18フレーム目で、高さも6桁まで同じ 0.310169。違いはそこから戻る量に出る。

粘着	いちばん低い高さ	戻った高さ	粘着 0 との比	40フレームの高さ
0.0	0.310169	0.026922	1.000000	0.336345
0.25	0.310169	0.024870	0.923792	0.334213
0.5	0.310169	0.015851	0.588767	0.324922
1.0	0.310169	0.013617	0.505791	0.321006

わかったこと

- ・Ground Friction は、そのままクーロン摩擦の係数だった。摩擦 0.25 で、式が言う距離 3.261978 に対し実測 3.260808。ずれは 0.04%。

- ・減速も $\mu \times g$ に合う。摩擦 0.1 のとき 1.625秒で速さが 1.592067 減り、1秒あたり 0.979733。 $\mu \times g = 0.981$ との差は 0.13%。

- ・距離は初速の2乗に比例する。初速 2.0 / 3.0 / 4.0 に対して「距離 $\div$ 初速²」が 0.202727 / 0.203753 / 0.203801。ほぼ一定。

- ・摩擦 0 なら、まったく減速しない。1.625秒で速さが減ったのは 0.000014 だけ。距離 6.499988 は初速 4.0×1.625 秒 = 6.5 とのずれが 0.000012。

- ・ただし摩擦を上げると式から外れる。0.5 で 7.4%、1.0 で 46.5%、2.0 では 2.8倍。塊が潰れて広がるからで、もう「滑る箱」ではなくなっている。

- ・ 潰れは数字に出ている。摩擦 0.0 では横の広がり が 0.995852、2.0 では 2.374637。高さは 0.502654 から 0.325032 へ。止まったのではなく、つぶれて広がった。

- ・ だから、ある所から先は摩擦を上げてても距離が縮まない。1.0 で 1.194665、2.0 で 1.148402。倍にして 4% しか変わらない。

- ・ Sticky は横滑りにはほとんど効かない。0.25 / 0.5 / 1.0 で距離 3.240815 / 3.240539 / 3.240539。0.5 と 1.0 は6桁まで同じ値。

- ・ Sticky が効くのは「離れるとき」。落として跳ねさせると、戻る高さが 0.026922 から 0.013617 へちょうど半分になる。いちばん低くなる高さは4通りとも 0.310169 で完全に同じ。押しつけている間は何もせず、離れようとした瞬間に引き止める数字である。

- ・ 測り方について。「式に合う／合わない」で終わらせず、合わなくなったときに何が起きているかを別の量で見た。距離だけを見ていたら「摩擦が強いと式から外れる」で終わっていたが、高さ と横の広がり を一緒に測っていたので「潰れている」と分かった。式が外れたら、外れた理由を測れる量に翻訳する。

次にやること

- ・ mpmcollider 側の friction / sticky も同じ式に乗るか
- ・ materialtype を変えると潰れ始める摩擦がどう動くか
- ・ mpmsolver の表面張力 (Surface Tension) を測る

MPM の材質5つで滑らせる — 摩擦の式に乗るのは、形が崩れない材質だけ

2026-09-24

何を作ったか

実験067で、Ground Friction はクーロン摩擦の μ そのものだと分かった。ただし摩擦が強いと塊が潰れて式から外れた。「潰れる」の中身は材質で決まっているはずなので、materialtype を5つとも試した。

摩擦 0.25 で、式が言う距離は 3.261978。Elastic は 3.246170 (式の 0.995154倍)、Chunky は 3.260808 (0.999642倍)。どちらも形を保ったまま滑り、式に乗る。ところが Viscous は 3.380790 (1.036423倍)、Sandy は 3.658329 (1.121506倍)、Liquid は 4.109069 (1.259686倍)。いずれも式より遠くまで行く。

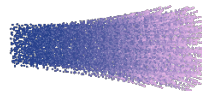
理由は奥行きを見れば分かる。元は 1辺 1.0 の箱なのに、40フレーム目の奥行きは Elastic 1.000850 / Chunky 1.000466 / Viscous 2.227771 / Sandy 5.863259 / Liquid 11.324130。横に広がった材質ほど、式から遠くへ外れている。広がるということは、塊の前側が後ろから押されて前へ出続けるということで、もう「1個の箱が滑って止まる」ではない。

摩擦を 1.0 にすると、5つとも式から大きく外れる (1.46倍~2.79倍)。Elastic の外れ方だけは理由が違った。高さの平均が $0.5376 \rightarrow 0.8331 \rightarrow 0.5952 \rightarrow 0.8523 \rightarrow 0.5874$ と3回上下しており、跳ねている。跳ねている間は地面に触れていないので摩擦が効かない。40フレーム目でもまだ 1.666963 の速さで動いていた。

もうひとつ、Liquid の摩擦 1.0 では計算が暴れた。25フレーム目以降、高さの平均は 0.0447 から 0.0563 までしか動かないのに、いちばん高い粒だけが $0.5760 \rightarrow 1.1335 \rightarrow 2.0569 \rightarrow 3.7952$ と上がり続ける。横の位置は 2.2173 から 2.2837 で止まっている。少数の粒が上へ飛んでいるだけで、物理ではなく数値の暴れと考えるのが自然である。この行の数字は結果として扱わない。

真上から見た跡 (摩擦 0.25)

1辺 1.0 の塊を $y = 0.52$ から横向きに初速 4.0 で投げた。40フレーム分を2フレームおきに重ねている。古いほど青く、新しいほど明るい。4枚ともカメラも枠も同じ。

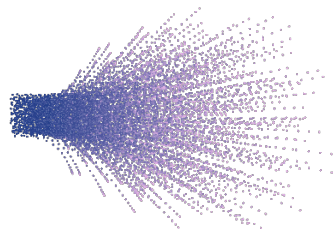


Houdini

Houdini

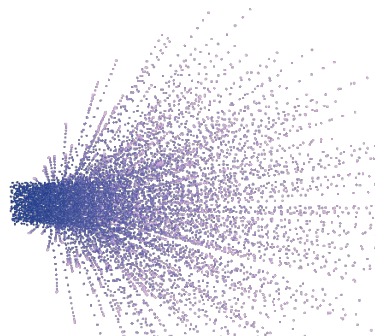
Chunky (既定)。奥行き 1.000466
で、幅がまったく変わらない。距離 3.260808 は式の
0.999642倍。

Viscous。奥行き 2.227771。距離 3.380790 は式の
1.036423倍。



Houdini

Sandy。奥行き 5.863259。距離 3.658329 は式の 1.121506倍。



Houdini

Liquid。奥行き 11.324130。距離 4.109069 は式の 1.259686倍。

A. 摩擦 0.25 — 式が言う距離は 3.261978

「奥行き」は元が 1.0。「潰れ具合」は 奥行き ÷ いちばん高い粒で、横に広がって縦に潰れるほど大きくなる。

材質	滑った距離	実測 ÷ 式	いちばん上	奥行き	潰れ具合
Elastic	3.246170	0.995154	1.010986	1.000850	0.9900
Chunky (既定)	3.260808	0.999642	0.983122	1.000466	1.0176
Viscous	3.380790	1.036423	0.290802	2.227771	7.6608
Sandy	3.658329	1.121506	0.063920	5.863259	91.7282
Liquid	4.109069	1.259686	0.046547	11.324130	243.2832

B. 摩擦 1.0 — 式が言う距離は 0.815494

Liquid のこの行は計算が暴れている（次の表を見よ）。数字は載せるが、結果としては扱わない。

材質	滑った距離	実測 ÷ 式	最後の vx	いちばん上	奥行き
Elastic	2.264021	2.776256	1.666963	1.270895	1.038518
Chunky (既定)	1.194665	1.464958	0.000409	0.647331	1.015006
Viscous	1.477935	1.812318	-0.003296	0.423962	1.727110
Sandy	1.676135	2.055360	-0.002204	0.275265	2.283884
Liquid	2.276151	2.791130	0.031310	3.795241	4.086964

Elastic が式から外れる理由（摩擦 1.0 のフレームごとの高さ）

跳ねている。地面に触れていない間は摩擦が効かないので、同じ摩擦でも遠くまで行く。

frame	x の平均	高さの平均	いちばん上
1	0.0075	0.5376	1.0156
7	0.6842	0.5971	1.1782

frame	x の平均	高さの平均	いちばん上
13	0.9213	0.8331	1.3811
19	1.1792	0.5952	1.2340
25	1.4839	0.8243	1.4976
31	1.7628	0.7363	1.3905
37	2.0616	0.6437	1.1369
40	2.2715	0.7196	1.2709

Liquid の摩擦 1.0 は暴れている（フレームごと）

高さの平均はほとんど動かないのに、いちばん高い粒だけが上がり続ける。横の位置も止まっている。少数の粒が上へ飛んでいるだけ。

frame	x の平均	高さの平均	いちばん上	奥行き
19	1.9928	0.0566	0.1523	2.7405
22	2.1322	0.0464	0.1418	2.8488
25	2.2173	0.0447	0.5760	2.8848
28	2.2568	0.0462	1.1335	2.8940
31	2.2704	0.0495	1.6652	2.8980
34	2.2758	0.0520	2.0569	3.1284
37	2.2798	0.0537	2.9850	3.5679
40	2.2837	0.0563	3.7952	4.0870

C. 形の崩れと、式からの外れを並べる（摩擦 0.25）

崩れる順に並べると、外れ方も同じ順になる。

材質	潰れ具合	実測 ÷ 式	奥行き
Elastic	0.9900	0.995154	1.000850
Chunky（既定）	1.0176	0.999642	1.000466
Viscous	7.6608	1.036423	2.227771
Sandy	91.7282	1.121506	5.863259
Liquid	243.2832	1.259686	11.324130

わかったこと

- ・【訂正・実験073】「Liquid は摩擦 0.25

では暴れない」は、40フレームで止めていたからだった。60フレームまで回すと、摩擦 0.25 でも46フレーム目から粒が飛び始める。暴れは摩擦 0 と 1.5 以上では起きず、substep 8 で消えた。表の数字（40フレーム目の値）はそのまま正しい。

- ・摩擦の式に乗るのは、形が崩れない材質だけだった。摩擦 0.25 で Elastic 0.995154倍、Chunky 0.999642倍。どちらも奥行きが 1.0008 / 1.0005 で、元の箱の幅のまま滑っている。

- ・崩れる順と、外れる順が完全に一致する。潰れ具合 0.99 / 1.02 / 7.66 / 91.7 / 243.3 に対して、実測÷式が 0.9952 / 0.9996 / 1.0364 / 1.1215 / 1.2597。例外は1つもない。

・崩れる材質は、必ず式より「遠くまで」行く。近くで止まるほうへは外れない。横に広がるということは前側が後ろから押され続けるということで、先頭はいつまでも前へ出る。

・Sandy は奥行きが 5.9倍、Liquid は 11.3倍になる。元は 1辺 1.0 の箱。真上から見た跡を並べると、同じ初速でもまったく違う絵になる。

・摩擦 1.0 では、5つとも式から大きく外れる（1.46倍～2.79倍）。実験067と同じで、摩擦を強くするほど式は使えなくなる。

・Elastic の外れ方だけは理由が別。跳ねている。高さの平均が $0.5376 \rightarrow 0.8331 \rightarrow 0.5952 \rightarrow 0.8523 \rightarrow 0.5874$ と3回上下する。地面に触れていない間は摩擦が効かないので、40フレーム目でもまだ 1.666963 の速さで動いていた。

・Liquid の摩擦 1.0 では計算が暴れた。25フレーム目以降、高さの平均は 0.0447 から 0.0563 までしか動かないのに、いちばん高い粒だけが $0.5760 \rightarrow 3.7952$ と上がり続ける。横の位置は 2.2173 から 2.2837 で止まっている。少数の粒が上へ飛んでいるだけなので、この行は結果として扱わない。

・「発散していないか」は、平均と最大を両方測れば分かる。平均が動かないのに最大だけが伸び続けたら、それは物理ではない。片方しか測っていないと、暴れを結果として書いてしまう。

・測り方について。実験067と同じ仕掛けを、軸だけ「摩擦」から「材質」に変えて走らせた。同じ物差しなので、67の表と68の表を直接並べられる。Chunky（既定の材質）の距離は、摩擦 0.25 で 3.260808、摩擦 1.0 で 1.194665。実験067で測った値と6桁まで完全に同じだった。

・ついでに、計算する範囲の大きさは結果を変えないと分かった。実験067では奥行き 8.0 の範囲、今回は 16.0 の範囲で組んだのに、同じ材質・同じ摩擦で距離が6桁まで一致した。中身が全部入っているかぎり、範囲を広く取っても結果は動かない。

次にやること

- ・mpmcollider 側の friction / sticky も同じ式に乗るか
- ・Liquid が暴れ始める摩擦の値を挟み撃ちで探す
- ・substep を上げると Liquid の暴れは収まるか

MPM のコライダの摩擦 — 式は同じ。ただし地面と重ねると完全に無視される

2026-09-24

何を作ったか

実験067で測ったのは mpm solver が最初から持っている地面だった。mpm collider にも friction と sticky があり、こちらは自分で置く物。同じものなのか、別物なのか。

まず組み込みの地面を切り、上面がちょうど $y = 0$ になる板を mpm collider として敷いて、摩擦を 0.0 から 1.0 まで振った。結果は組み込みの地面とほぼ同じ。摩擦 0.0 で 6.499993 対 6.499988 (差 0.000005)、0.25 で 3.250318 対 3.260808 (比 0.996783)、1.0 で 1.187338 対 1.194665 (比 0.993867)。いちばん離れたところで 0.6% しか変わらない。クーロン摩擦の式との比も 0.996426 (摩擦 0.25) で、実験067の 0.999642 と同じ水準だった。摩擦の扱いは同じものと考えてよい。

ところが両方を同時に置くと、話が変わる。地面の摩擦 0.0・コライダの摩擦 1.0 にしても、距離は 6.499988。これは摩擦がまったく無い場合の値と6桁まで同じで、つまりコライダは何もしていない。逆に地面 1.0・コライダ 0.0 では 1.193125 で、地面だけのときの 1.194665 とほぼ同じ。同じ高さに重ねると、組み込みの地面が完全に勝つ。

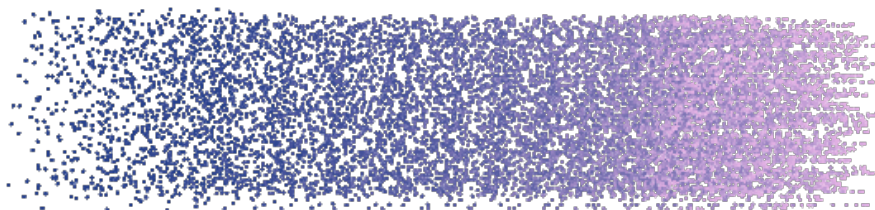
「地面がいつも勝つ」のか、「先に触ったほうが勝つ」のかを分けるため、地面だけ $y = -0.5$ に下げてみた。すると逆転する。地面 1.0・コライダ 0.0 で 6.499993 (コライダだけ 0.0 のときと同じ)、地面 0.0・コライダ 1.0 で 1.187338 (コライダだけ 1.0 のときと6桁まで同じ)。効くのは先に触った面だけで、その下にある面は無いのと同じだった。

実用上の意味ははっきりしている。床を自分で置くなら、組み込みの地面を切る。切らずに同じ高さへ置くと、コライダの摩擦をいくら設定しても効かない。しかもエラーも警告も出ない。

ただし「コライダが無いのと同じ」というわけではなかった。実験062~065と同じ落とし方 ($y = 3.0$ から落とす) で床の箱ありと無しを測り直すと、高さの平均が 0.303183 と 0.304588 で 0.46% 違う。摩擦は効かないが、当たる面としては働いている。ついでに、床ありの 0.303183 は実験064・065で測った値と6桁まで完全に一致した。

コライダの上を滑らせた跡 (摩擦 0.25)

組み込みの地面は切ってある。上面がちょうど $y = 0$ になる板を mpm collider として敷いた。40フレーム分を2フレームおきに重ねている。



Houdini

コライダの摩擦 0.25。滑った距離 3.250318。組み込みの地面での 3.260808 と 0.3% しか変わらない。

A・B. コライダと組み込みの地面を、同じ摩擦どうしで並べる

「組み込みの地面」の列は実験067で測った値。条件は初速 4.0、40フレーム、粒の間隔 0.12
でまったく同じ。摩擦 0.1 はどちらもまだ止まっていないので、式との比は意味を持たない。

摩擦 μ	コライダ	組み込みの地面	差	コライダ ÷ 地面	式の距離	コライダ ÷ 式
0.0	6.499993	6.499988	0.000005	1.000001	∞	—
0.1	5.199496	5.204091	-0.004595	0.999117	8.154944	まだ止まっていない
0.25	3.250318	3.260808	-0.010490	0.996783	3.261978	0.996426
0.5	1.748732	1.751774	-0.003042	0.998264	1.630989	1.072191
1.0	1.187338	1.194665	-0.007327	0.993867	0.815494	1.455974

C. 両方を同じ高さ ($y = 0$) に置く

3行目に注目。コライダの摩擦を 1.0 にしても、摩擦が無いときの 6.499988 とまったく同じ値になる。

地面の摩擦	コライダの摩擦	滑った距離	最後の vx	何と同じ値か
0.25	0.25	3.260318	0.025391	地面だけ 0.25 (3.260808) とほぼ同じ
1.0	0.0	1.193125	0.000432	地面だけ 1.0 (1.194665) とほぼ同じ
0.0	1.0	6.499988	3.999986	摩擦なし (6.499988) と6桁まで同じ
0.25	コライダなし	3.260808	0.025688	実験067の値そのもの

E. 実験062～065と同じ組み方で、床の箱があってもなくても同じか

1辺 1.0 の塊を $y = 3.0$ から落として30フレーム目。床の箱は size (6, 0.4, 6) を t (0, -0.2, 0) に置いたもので、上面がちょうど $y = 0$ 。組み込みの地面はどちらも入れたまま。

床の箱	粒	高さの平均	いちばん上	広がり	$J_e \times J_p$
あり	577	0.303183	0.663563	1.090555	0.689148
なし	577	0.304588	0.665211	1.090218	0.689286
差	0	0.001404	0.001648	0.000336	0.000138

D. 地面だけ $y = -0.5$ に下げて、先に触るのをコライダにする

groundposy を -0.5 にした。コライダの板は同じ位置（上面が $y = 0$ ）のまま。

地面の高さ	地面の摩擦	コライダの摩擦	滑った距離	何と同じ値か
-0.5	1.0	0.0	6.499993	コライダだけ 0.0 (6.499993) と完全に同じ
-0.5	0.0	1.0	1.187338	コライダだけ 1.0 (1.187338) と6桁まで同じ
-0.5	1.0	コライダなし	1.871450	0.5 落ちてから滑るので、その分だけ遠い

わかったこと

- ・コライダの摩擦は、組み込みの地面と同じものだった。5通りの摩擦で、いちばん離れたところでも 0.6% しか変わらない。摩擦 0.0 では差が 0.000005。
- ・クーロン摩擦の式との合い方も同じ水準。摩擦 0.25 で 0.996426（実験067の組み込み地面は 0.999642）。強い摩擦で式から外れていく癖も同じで、0.5 で 1.072191、1.0 で 1.455974。
- ・ただし両方を同じ高さに置くと、コライダは完全に無視される。地面の摩擦 0.0・コライダの摩擦 1.0 で、距離は 6.499988。摩擦がまったく無いときの値と6桁まで同じ。
- ・エラーも警告も出ない。コライダはちゃんとつながっていて、粒もその上に乗っている。それでも摩擦だけが効かない。
- ・「地面がいつも勝つ」のではなく「先に触った面が勝つ」だった。地面を $y = -0.5$ に下げると、コライダ 0.0 で 6.499993、コライダ 1.0 で 1.187338。どちらもコライダだけを置いたときの値と一致する。下にある面は無いのと同じ。
- ・だから、床を自分で置くなら組み込みの地面を切る。groundactive を 0 にするだけ。切り忘れると、コライダの摩擦をいくら設定しても効かない。
- ・ただし「コライダが無いのと同じ」ではない。実験062～065と同じ落とし方で、床の箱ありと無しを測り直した。高さの平均は 0.303183 と 0.304588 で、0.46% だけ違う。摩擦は効かないが、当たる面としては働いている。
- ・実験062～065の数字はそのまま有効。床の箱ありで測り直した高さ 0.303183 は、実験064（substep 1）・実験065（間隔 0.12）の値と6桁まで完全に同じだった。広がり 1.090555 も実験064の 1.09055 と一致する。別の日に別の目的で組み直した実験が同じ数字を返したので、あちらの測定も再現できている。
- ・測り方について。「同じか違うか」を確かめるとき、まず片方だけで測り、次にもう片方だけで測り、最後に両方を置いた。両方を置いた場合だけを見ていたら「コライダの摩擦は効かない」で終わっていた。高さをずらす一手間で、「効かない」ではなく「先に触った面だけが効く」という一段深い答えになった。
- ・推測を書かないことについて。「摩擦が効かないのだから、あの床の箱は無くても同じ結果だったはずだ」と書きかけて、実際に測った。違った（0.46%）。筋が通っている推論でも、測れるなら測る。

次にやること

- ・コライダの sticky も、地面の sticky と同じか
- ・動くコライダ（computevelocity）は粒にどれだけ速度を渡すか
- ・Liquid が暴れ始める摩擦の値を挟み撃ちで探す

動く板は塊をどれだけ運ぶか — 追いつくまでは式どおり。追いついた先で潰れて遅れる

2026-09-24

何を作ったか

実験069でコライダの摩擦が組み込みの地面と同じものと分かったので、今度は板のほうを動かす。ベルトコンベアの上に物を置いたようなもの。

まず組んだら、板をいくら早くしても塊が1ミリも動かなかった。5通りの速さで結果が6桁まで完全に同じ。mpmcollider の Type が既定で Static だからで、Animated (Rigid) に変えて初めて動き出す。エラーも警告も出ない。実験044のガイド、050の可変長入力、062の materialpreset、066の風速に続いて、これで5件目になる。

動かしてからは素直だった。板の速さ 1 / 2 / 4 / 8 に対して、40フレーム目の塊の速さは 1.049126 / 1.993989 / 3.998951 / 8.024615。板の速さで割ると 1.049 / 0.997 / 1.000 / 1.003 で、きちんと板に追いついている。

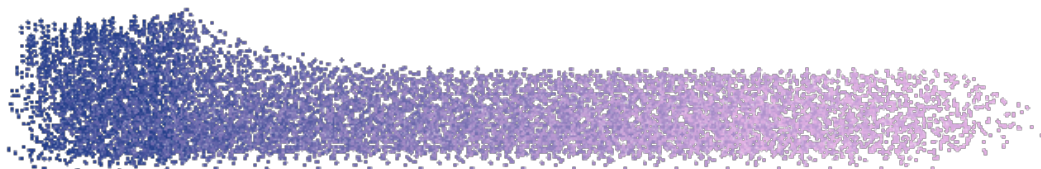
運ばれた距離は2段階になる。追いつくまでは「 $\frac{1}{2} \mu g t^2$ 」、追いついたあとは「板の速さ × 時間 - 板の速さ² ÷ (2 μg)」。摩擦を振って確かめた。まだ追いついていない摩擦 0.1 では式 1.295227 に対し実測 1.300902 (比 1.004382)、0.25 では 3.238066 に対し 3.248055 (比 1.003085)。0.4% 以内で合う。

ところが摩擦を上げると足りなくなる。0.5 で 0.974643、1.0 で 0.938715、2.0 で 0.879849。理由は高さを見れば分かる。摩擦 0 (運ばれない) で 0.501119 だった高さが、摩擦 1.0 では 0.338196、板の速さ 8 では 0.251712 まで下がる。塊が横向きにこすられて潰れており、底は板と一緒に動いても上のほうが遅れている。平均で測っているのだから、そのぶん短く出る。

computevelocity についても分かった。Animated (Rigid) では入れても切っても6桁まで同じ値で、まったく関係ない。ところが Animated (Deforming) では、切ると塊が動かなくなる (0.001513 で Static と同じ値)。形が変わる物を動かすときだけ必要な項目だった。

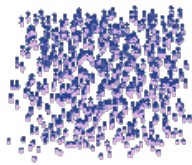
板を動かしたときと、止めたとき

1辺 1.0 の塊を、上面が $y = 0$ の板の上に置いただけ。塊には初速を与えていない。40フレーム分を2フレームおきに重ねている。古いほど青く、新しいほど明るい。2枚とも摩擦 1.0、カメラも枠も同じ。



Houdini

板の速さ 4.0。5.336132 運ばれる。先へ行くほど薄く伸びているのは、横にこすられて潰れているから (高さ 0.501119 → 0.338196)。



板の速さ 0。40フレーム分がすべて同じ場所に重なる。運ばれた距離 0.001643。

C. Type と computevelocity (板の速さ 4.0・摩擦 1.0)

1行目と5行目は、板を止めたときとまったく同じ値になっている。

Type	computevelocity	運ばれた距離	最後の vx	高さ
Static (既定)	入り	0.001513	0.001907	0.501120
Animated Rigid	入り	5.336132	3.998951	0.338196
Animated Rigid	切り	5.336132	3.998951	0.338196
Animated Deforming	入り	5.335937	3.998780	0.338201
Animated Deforming	切り	0.001513	0.001907	0.501120

A・B. 板の速さと、運ばれた距離 (摩擦 1.0)

経過時間 1.625 秒。「追いつく秒」は 板の速さ ÷ (μg)。4通りとも 1.625 秒より短いので、式は「板の速さ × 時間 - 板の速さ² ÷ ($2 \mu g$)」のほう。

板の速さ	運ばれた距離	式の距離	実測 ÷ 式	最後の vx	vx ÷ 板の速さ	高さ	追いつく秒
0.0	0.001643	0.000000	—	0.001990	—	0.501120	∞
1.0	1.516453	1.574032	0.963420	1.049126	1.049126	0.494549	0.1019
2.0	2.776795	3.046126	0.911582	1.993989	0.996995	0.400620	0.2039
4.0	5.336132	5.684506	0.938715	3.998951	0.999738	0.338196	0.4077
8.0	9.479770	9.738022	0.973480	8.024615	1.003077	0.251712	0.8155

D. 摩擦を変える (板の速さ 4.0)

摩擦 0.1 と 0.25 はまだ板に追いついていないので、式は「 $\frac{1}{2} \mu g t^2$ 」のほう。0.5 以上は追いついたあとの式を使う。

摩擦	運ばれた距離	式の距離	実測 ÷ 式	最後の vx	追いつく秒
0.0	-0.000000	0.000000	—	-0.000000	∞
0.1	1.300902	1.295227	1.004382	1.597380	4.0775
0.25	3.248055	3.238066	1.003085	3.991266	1.6310

摩擦	運ばれた距離	式の距離	実測 ÷ 式	最後の vx	追いつく秒
0.5	4.745549	4.869011	0.974643	3.972947	0.8155
1.0	5.336132	5.684506	0.938715	3.998951	0.4077
2.0	5.360264	6.092253	0.879849	4.002565	0.2039

潰れ具合と、式からの足りなさを並べる

板の速さを変えた4通り（摩擦 1.0）。速い板ほど強くこすられ、そのぶん平均が遅れる。

板の速さ	高さ	摩擦0のときからの減り	実測 ÷ 式
0.0	0.501120	0.0%	—
1.0	0.494549	1.3%	0.963420
2.0	0.400620	20.1%	0.911582
4.0	0.338196	32.5%	0.938715
8.0	0.251712	49.8%	0.973480

わかったこと

- ・ mpmcollider の Type は既定が Static。板をいくら動かしても、塊は何も感じない。5通りの速さで結果が6桁まで完全に同じだった。Animated (Rigid) に変えて初めて動き出す。エラーも警告も出ない。
- ・ 「黙って無視されるパラメータ」はこれで5件目。実験044のガイド、050の可変長入力、062の materialpreset、066の風速、そして今回の Type。見つけ方はいつも同じで、入力を極端に振って出力が動くかを見る。
- ・ 動かせば、塊はきちんと板の速さに追いつく。板 1 / 2 / 4 / 8 に対して塊の速さが 1.049126 / 1.993989 / 3.998951 / 8.024615。板で割ると 1.049 / 0.997 / 1.000 / 1.003。
- ・ 追いついていない間は、式に 0.4% で乗る。摩擦 0.1 で式 1.295227 に対し実測 1.300902、0.25 で 3.238066 に対し 3.248055。「 $\frac{1}{2} \mu g t^2$ 」そのもの。
- ・ 追いついたあとは足りなくなる。摩擦 0.5 で 0.974643、1.0 で 0.938715、2.0 で 0.879849。摩擦を強くするほど式から遠ざかる。
- ・ 理由は潰れ。高さが 0.501119 → 0.494549 → 0.400620 → 0.338196 → 0.251712。板が速いほど強くこすられ、底は板と一緒に動いても上のほうが遅れる。平均で測っているのに、そのぶん短く出る。
- ・ 摩擦 0 の板は、本当に何も運ばない。運ばれた距離 -0.000000、最後の vx も -0.000000。動いている板の上でも、摩擦が無ければ物は置き去りになる。
- ・ computevelocity は Rigid では無関係だった。入れても切っても6桁まで同じ値。ところが Animated (Deforming) では、切ると塊が動かなくなる（0.001513 で Static と同じ値）。形が変わる物を動かすときだけ必要な項目。
- ・ Rigid と Deforming は、剛体の動きに対しては同じ答えを出す。5.336132 対 5.335937。差は 0.000195。剛体で済むなら Rigid でよい。
- ・ 測り方について。式が2段階に分かれることを先に整理してから測った。「追いつく秒」を計算しておけば、どの行にどちらの式を当てるかで迷わない。ひとつの式を全部の行に当ててしまうと、合わない行が「外れた」ように見えてしまう。

次にやること

- 板を回してみる (Animated Rigid の回転は速度としてどう伝わるか)
- 形が変わるコライダ (Animated Deforming) を実際にたわませる
- Liquid が暴れ始める摩擦の値を挟み撃ちで探す

回る台は塊をどこまで乗せておけるか — 境目は式の形どおり。ただし摩擦は 0.3~0.4 ぶんしか効かない

2026-09-24

何を作ったか

実験070では板をまっすぐ動かした。今度は台を回す。レコード盤の上に物を置いたようなもの。

回る台の上の物が一緒に回るには、中心へ向かう力 $m \omega^2 r$ が要る。それを出せるのは摩擦だけで、上限は $\mu m g$ 。だから $\omega^2 r$ が μg を超えたところで外へ滑り出すはず。半径 2・摩擦 1.0 なら、境目は $\omega = \sqrt{(9.81 \div 2)} = 2.215 \text{ rad/秒}$ 。

まず回転そのものは正確に伝わった。台に乗っている間の塊の回る速さは、台の 0.98~1.00 倍。Animated (Rigid) のコライダは、回転もきちんと速度として粒に渡している。

ところが境目は外れた。半径 2 では ω 1.25 で乗ったまま、1.30 で滑り出した。式の 2.215 の 0.58 倍しかない。

式の形は合っていた。半径を 1 / 2 / 3 / 4 と変えて境目を挟み撃ちで探すと、 $\omega^2 r$ が 3.71 / 3.25 / 3.47 / 3.06 とほぼ一定になった。半径が倍なら境目の ω は $\sqrt{2}$

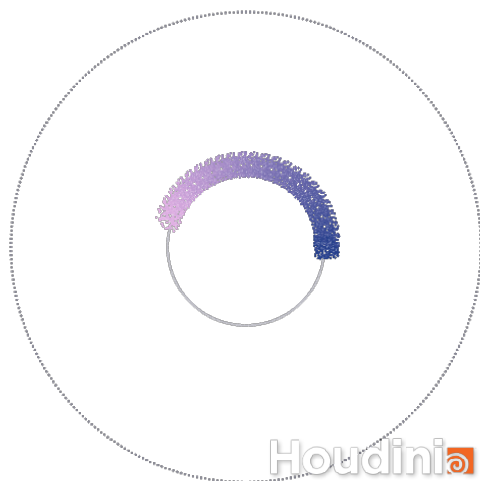
分の1、という形はそのまま出ている。違うのは定数だけで、 $\mu g = 9.81$ のところが 3.06~3.71。摩擦に直すと 0.31~0.38 ぶんしか効いていない。

摩擦を変えると、弱い摩擦ではほぼ式どおりだった。摩擦 0.25 で境目 ω は式の 0.95倍、0.5 で 0.90倍。ところが 1.0 で 0.58倍、2.0 で 0.47倍。0.5 より上は、摩擦を上げてても実効の値が 0.33~0.44 から上がらない。実験067・069で「強い摩擦ほど式から外れる」と見ていたものが、ここではっきり上限として出た。

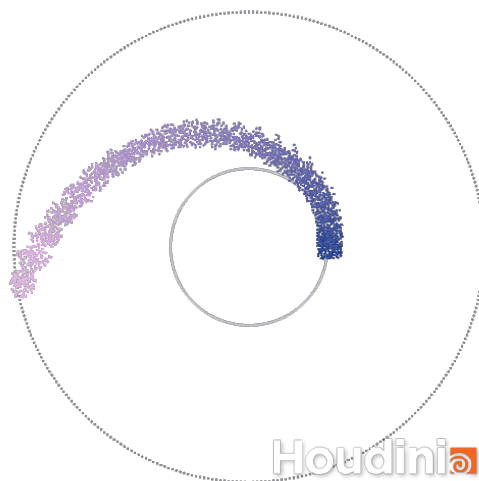
境目より下でも、塊はゆっくり外へずれる。 ω 1.0 で 3秒に 0.039、1.25 で 0.149。止まっているのではなく、少しずつ滑っている。

境目の下と上

1辺 0.6 の塊を半径 2 に置き、台を回した（摩擦 1.0）。3秒ぶんを3フレームおきに重ね、真上から撮った。古いほど青く、新しいほど明るい。内側の円が半径 2、外側の円が台の縁（半径 6）。2枚とも同じカメラ。



ω 1.0。台と一緒に回る（塊の回る速さは台の 0.9993倍）。3秒で外へ 0.039 だけずれた。



ω 1.5。回りきれずに外へ滑り出し、3秒で半径 5.94 まで出た。式の境目 2.215 より、ずっと遅い回転で滑った。

A. 半径 2 で、回す速さを変える（摩擦 1.0。式の境目は ω 2.215）

3秒（72フレーム）回したあとの値。「塊の $\omega \div$ 台の ω 」は、台に乗っていれば 1 になる。

台の ω	外へずれた距離	塊の $\omega \div$ 台の ω	塊が回った角度	台が回った角度	計算の秒
0.0	0.0027	—	0.0000	0.0000	3.61
0.25	0.0039	0.9851	0.7252	0.7396	2.06
0.5	0.0072	0.9827	1.4244	1.4792	2.37
0.75	0.0144	1.0013	2.1518	2.2188	2.70
1.0	0.0394	0.9993	2.8288	2.9583	2.91
1.25	0.1488	1.0000	3.5217	3.6979	2.98
1.5	3.9350	0.5760	3.3684	4.4375	2.90
2.0	6.7892	0.2279	3.4729	5.9167	2.98
2.4	11.0868	0.0911	3.3178	7.1000	2.91
3.0	15.5046	0.0437	2.9058	8.8750	2.89

B. 置く半径を変えて、境目を挟み撃ちで探す（摩擦 1.0）

3秒で半径が 1.0 以上外へ動いたら「滑り出した」とした。6回の挟み撃ちで幅 0.05 まで詰めた。式どおりなら $\omega^2 r$ は 9.81 で一定。

半径	境目の ω	式の境目	実測 $\div$ 式	$\omega^2 r$	実効の摩擦
1.0	1.90~1.95	3.1321	0.6146	3.7056	0.3777
2.0	1.25~1.30	2.2147	0.5757	3.2512	0.3314
3.0	1.05~1.10	1.8083	0.5945	3.4669	0.3534
4.0	0.85~0.90	1.5660	0.5587	3.0625	0.3122

C. 摩擦を変えて、境目を探す（半径 2）

摩擦 1.0 の行は B と同じもの。式どおりなら「実効の摩擦」は入れた摩擦と同じ値になる。

入れた摩擦	境目の ω	式の境目	実測 ÷ 式	実効の摩擦
0.25	1.025~1.077	1.1074	0.9489	0.2251
0.5	1.386~1.438	1.5660	0.9015	0.4063
1.0	1.25~1.30	2.2147	0.5757	0.3314
2.0	1.438~1.489	3.1321	0.4672	0.4365

わかったこと

- ・回転はきちんと伝わる。台に乗っている間、塊の回る速さは台の 0.98~1.00 倍 (ω 0.25~1.25)。Animated (Rigid) のコライダは、平行移動（実験070）だけでなく回転も速度として粒に渡している。
- ・境目は式よりずっと早く来た。半径 2・摩擦 1.0 で、式の境目 ω 2.215 に対して、実際は 1.25~1.30。0.58倍。
- ・式の形は合っている。半径 1 / 2 / 3 / 4 で $\omega^2 r$ が 3.71 / 3.25 / 3.47 / 3.06 とほぼ一定（平均 3.37、ばらつき $\pm 10\%$ ）。「半径が大きいほど、ゆっくり回しても滑る」という向きと比率はそのまま出ている。違うのは定数で、 $\mu g = 9.81$ のところが 3.1~3.7。
- ・摩擦は、0.3~0.4 ぶんしか効いていない。摩擦 0.25 で実効 0.225、0.5 で 0.406 と、弱いうちはほぼ入れたとおり。ところが 1.0 で 0.331、2.0 で 0.437。0.5 より上は、摩擦を上げてても実効の値が上がらない。
- ・実験067・069の「強い摩擦ほど式から外れる」と同じ話だった。滑り落ちる塊（067）や動く板（070）では「強い摩擦で少しずれる」程度に見えていたが、止まっていられるかどうかを問うと、はっきり上限として出る。
- ・境目の下でも、塊はゆっくり外へずれる。 ω 1.0 で3秒に 0.039、1.25 で 0.149。回っていない台 (ω 0) では 0.0027。完全に止まっているのではなく、少しずつ滑っている。
- ・滑り出すと、塊はもう台に追いつかない。 ω 3.0 では塊の回る速さが台の 0.044倍。外へ飛ばされながら、ほとんど回らなくなる。
- ・実用の結論。MPM で「回る台に物を乗せておく」なら、摩擦の値ではなく回す速さで調整する。摩擦 1.0 を超えて上げてても、乗せておける限界はほとんど延びない。目安は $\omega^2 r < 3$ （この場面の粒の細かさ 0.12 で）。
- ・測り方について。「滑り出した」を 3秒で半径が 1.0 以上外へ動いたか で決めた。境目の下でもゆっくりずれるので、この決め方しだいで境目は少し動く。ただ、境目の上ではずれが 1.6~13.6 と桁違いに大きく、決め方に迷う例はなかった。
- ・計算の時間は、108粒・72フレーム（3秒）で 1回 2.1~4.5秒。計算範囲は $16 \times 4 \times 16$ 。挟み撃ちの1点ごとに作り直している。

次にやること

- ・粒の細かさ (particlesep) を変えると、実効の摩擦の上限は動くか
- ・形が変わるコライダ (Animated Deforming) を実際にたわませる
- ・Liquid が暴れ始める摩擦の値を挟み撃ちで探す

たわむ板は塊を放り上げられるか — Deforming は板の速さを渡す。Rigid はたわみを平均の動きにってしまう

2026-09-24

何を作ったか

実験070では「形が変わらない板」を動かした。今度は板そのものを曲げる。細かく割った板の真ん中を、決まった速さ v で高さ 1.0

まで持ち上げて止める。上に乗った塊は、板が止まった瞬間に放り出されるはず。

投げ上げの式どおりなら、放り出されたあと、さらに $v^2 \div (2g)$ だけ上がる。 $v = 2 / 4 / 8$ なら $0.204 / 0.816 / 3.262$ 。

Animated (Deforming) ・ computevelocity

入りでは、塊はきちんと放り上げられた。持ち上げている間の塊の上向きの速さは、最大で $1.17 / 2.19 / 4.28 / 8.00$ ($v = 1 / 2 / 4 / 8$) と、板の速さにそろう。ただし上がった高さは式に届かず、 $0.061 / 0.506 / 2.473$ で、式の $30\% / 62\% / 76\%$ 。速く持ち上げるほど式に近づき、 $v = 1$ では板から離れなかった。

Type を変えると、まったく別の振る舞いになった。Static は070と同じく何も伝えない。Animated (Rigid) は、たわみを「板全体の平均の持ち上がり」として扱った。塊の上向きの速さは 2.50

で、真ん中の速さ 4 ではなく、板の平均の速さ $4 \times 2/n = 2.55$

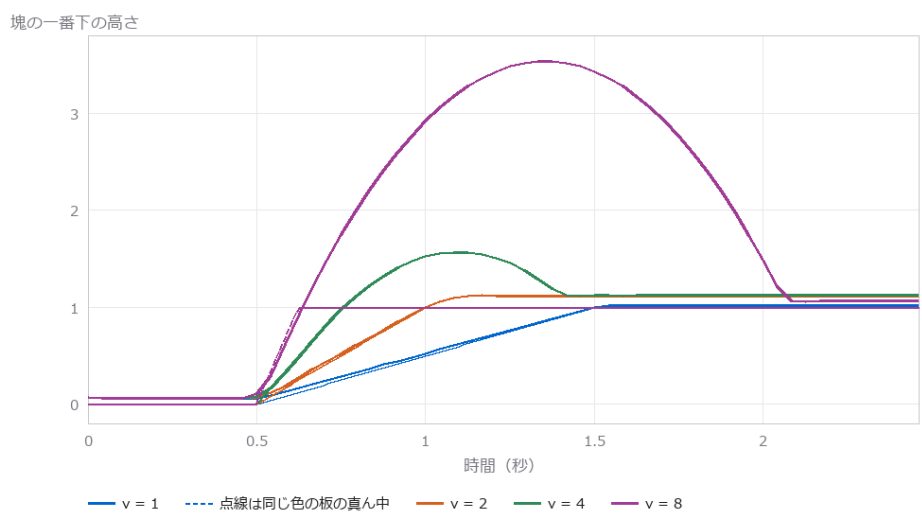
にほぼ一致する。最後に塊が落ち着いた高さも 0.647 で、平均の持ち上がり 0.637 に近い。

Deforming で computevelocity を切ると、板が動き出した瞬間に塊が板をすり抜けて落ちた。1.54秒で $y = -3.69$ 、下向きの速さ 8.5 で、そのまま自由落下している。実験070では「切ると塊が動かない」だったが、板が塊に向かって動く場合は、すり抜けになる。

計算の時間も違う。同じ場面 ($v = 4$) で Static 3.05秒、Rigid 3.83秒、Deforming 8.98秒。形が変わるコライダは、Rigid の2.3倍かかる。

塊の高さの動き (Deforming ・ computevelocity 入り)

1辺 0.6 の塊を板の真ん中に置き、0.5秒から板の真ん中を持ち上げた。実線が塊の一番下の高さ、同じ色の点線が板の真ん中の高さ。塊は板の上面より 0.057 上に浮いて乗っている (コライダの厚み)。



v = 8 は板が止まった 0.625秒から 0.71秒上がり続け、3.53 まで届いた。v = 1 は板から離れずに一緒に上がっただけ。

A. 持ち上げる速さを変える (Deforming・computevelocity 入り・摩擦 1.0)

「板より上」は、塊の一番下の最高点から、止まった板の上面（1.0 + 浮き 0.057）を引いたもの。1/24秒ごとに見ているので、最高点は最大 1フレームぶんずれる。

v	板が止まる秒	塊の最大の上向きの速さ	止まる直前の上向きの速さ	板より上に上がった高さ	式 $v^2 \div 2g$	実測 ÷ 式	計算の秒
1	1.500	1.1689	0.7151	-0.0417	0.0510	—	7.27
2	1.000	2.1866	1.5678	0.0613	0.2039	0.301	7.74
4	0.750	4.2798	3.4746	0.5059	0.8155	0.620	9.14
8	0.625	7.9968	7.3339	2.4728	3.2620	0.758	7.97

B. v = 4 で、Type と computevelocity を変える

Rigid の2行は6桁まで同じ値。Deforming で切った行は、板をすり抜けて落ちている（最高点が止まる前の 0.75秒にあり、そのあと下がり続ける）。

Type	computevelocity	止まる直前の上向きの速さ	塊の一番下の最高点	計算の秒
Static (既定)	入り	-0.0041	0.0566	3.05
Animated (Rigid)	入り	2.5004	0.9379	3.83
Animated (Rigid)	切り	2.5004	0.9379	3.88
Animated (Deforming)	入り	3.4746	1.5630	8.98
Animated (Deforming)	切り	-0.7375	-0.0054	8.84

わかったこと

- Deforming・computevelocity 入りなら、板のたわみは速さとして塊に渡る。持ち上げている間の塊の上向きの速さは、最大で 1.17 / 2.19 / 4.28 / 8.00。板の真ん中の速さ 1 / 2 / 4 / 8 にそろろう。

- ・ただし放り上げの高さは式に届かない。板より上に $0.061 / 0.506 / 2.473$ ($v = 2 / 4 / 8$) で、式 $v^2 \div 2g$ の 30% / 62% / 76%。速く持ち上げるほど式に近い。 $v = 1$ では板から離れなかった（式でも 0.051 しか上がらない）。

- ・放り出されたときの速さが、もう板の速さより小さい。 $v = 8$ で、板が止まってから最高点までの時間 0.71秒に g をかけると 6.95 (v の 0.87倍)。この速さで上がる高さ 2.46 は実測 2.473 と合う。つまり足りない分は、飛んでいる間ではなく、放り出される前に失われている。なぜかは確かめていない。

- ・Animated (Rigid) は、たわみを「板全体の平均の動き」として扱う。塊の上向きの速さは 2.50。真ん中の速さ 4 ではなく、 $\cos$ の山の平均 $4 \times 2/n = 2.55$ に近い。落ち着いた高さ 0.647 も、平均の持ち上がり $2/n = 0.637$ に近い。曲がる板を Rigid にすると、形の変化が消えて平均の上下動だけが残る。

- ・Rigid では computevelocity は無関係。入り・切りで6桁まで同じ値だった（実験070と同じ）。

- ・Deforming で computevelocity を切ると、塊は板をすり抜けて落ちる。板が動き出した 0.6秒ごろから下がり始め、1.54秒で $y = -3.69$ ・下向きの速さ 8.5。自由落下そのもの。エラーも警告も出ない。実験070（板が横に動く）では「切ると動かない」だったが、板が塊に向かって動くと、すり抜けになる。

- ・Static は何も伝えない。板をいくら曲げても、塊は最初の高さ 0.057 のまま（実験070と同じ）。

- ・形の変わるコライダは重い。同じ場面で Static 3.05秒、Rigid 3.83秒、Deforming 8.98秒。Deforming は Rigid の2.3倍。板が曲がらないなら Rigid で足りる（実験070でも、剛体の動きなら2つの答えは6桁近くまで同じだった）。

- ・測り方について。高さは「塊の一番下の点」で測った。平均の高さでは、塊が潰れたり伸びたりした分が混ざってしまう。また塊は板の上面より 0.057 上に浮いて乗っていたので、「板より上」はその分を引いてある。

次にやること

- ・止まる直前に減速するのはなぜか（粒の細かさ・substep を変えて見る）
- ・Liquid が暴れ始める摩擦の値を挟み撃ちで探す
- ・コライダの sticky も、組み込みの地面の sticky と同じか

液体はいつ暴れ出すか —

摩擦の境目ではなく「時間」だった。substep 8で消えるが 5倍かかる

2026-09-24

何を作ったか

実験068で、Liquid を摩擦 1.0 の地面で滑らせたなら計算が暴れた。高さの平均は 0.05

前後なのに、いちばん高い粒だけが上がり続けた。摩擦 0.25

では暴れなかったので、境目はこの間にあると考えて、摩擦を細かく振った。

見立ては外れた。068は40フレームで止めていたが、今回は60フレームまで回した。すると摩擦 0.25

でも、50フレーム目から粒が飛び始めた。0.4 / 0.5 / 0.6 / 0.7 / 0.8 / 0.9 / 1.0

も、すべて60フレームまでに暴れた。

120フレームまで回して、暴れ始めるフレームを測った。摩擦 0.25 で F46、0.5 で F30、1.0 で

F28。摩擦が強いほど早く始まる。ところが摩擦 1.5 / 2.0 / 3.0

では、120フレームまで1粒も飛ばなかった。摩擦 0（滑りっぱなし）も静か。暴れるのは「0 より大きく 1.5 より小さい」摩擦のときだけだった。

飛ぶのは 547粒のうち 1~12粒。高さの平均は 0.03~0.13

で、液そのものは地面に広がったまま。いちばん高い粒は 20

近くまで上がる。物理ではなく数値の暴れと考えるのが自然である。

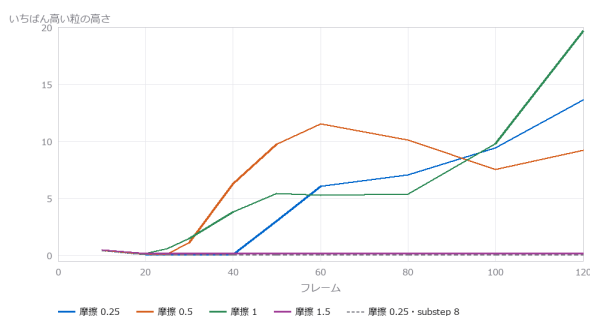
substep を上げると消えた。摩擦 1.0 で substep 1 / 2 / 4 はまだ暴れ、8 で静かになった。摩擦 0.25

も substep 8 と 16 で120フレームまで静か。ただし時間は、60フレームで 2.82秒 →

13.16秒（4.7倍）、120フレームで 5.36秒 → 33.85秒（6.3倍）かかる。

いちばん高い粒の高さ（120フレーム）

1辺 1.0 の Liquid の箱を、初速 4 で組み込みの地面に滑らせた（実験068と同じ組み方）。印を付けたフレームだけ測っているの、線は折れ線になる。



摩擦 0.25 は F46、0.5 は F30、1.0 は F28

から上がり始める。摩擦 1.5 と、摩擦 0.25・substep 8

は、ずっと 0.2 以下。



摩擦 1.0 の60フレーム目。液は地面に広がっているのに、数粒だけが宙に浮いている（飛んだ粒まで入る枠で撮った）。

A. 摩擦を振る（60フレーム・既定の substep 1）

判定は「高さの平均が 0.2 未満なのに、いちばん高い粒が 1.0 を超える」。

摩擦	判定	高さの平均	いちばん高い粒	0.5 より上の粒	計算の秒
0.25	暴れた	0.0406	6.0646	11 / 547	2.84
0.4	暴れた	0.0715	11.4444	9 / 547	2.12
0.5	暴れた	0.0576	11.4944	6 / 547	2.06
0.6	暴れた	0.0324	5.8894	3 / 547	2.26
0.7	暴れた	0.0554	14.5527	3 / 547	2.50
0.8	暴れた	0.0397	6.6924	1 / 547	2.27
0.9	暴れた	0.0396	2.3328	2 / 547	2.56
1.0	暴れた	0.0602	5.2790	5 / 547	2.69
1.5	静か	0.0530	0.1631	0 / 547	2.70

B. 120フレームまで回して、暴れ始めるフレームを見る

「始まり」は、いちばん高い粒が初めて 1.0 を超えたフレーム。摩擦 0 は地面を滑り続け、120フレームで $x = 19.8$ まで行った。

摩擦	始まり	120フレーム目の高さの平均	いちばん高い粒	計算の秒
0	なし	-0.0443	-0.0412	6.50
0.25	F46	0.1300	13.5952	5.22
0.5	F30	0.0438	9.2167	6.03
1.0	F28	0.0829	19.6697	5.93
1.5	なし	0.0491	0.1692	6.57
2.0	なし	0.0690	0.1849	6.56
3.0	なし	0.0807	0.2010	6.26

C・D. substep を上げる

摩擦 1.0 は60フレーム、摩擦 0.25 は120フレームまで回した。substep 1 が既定。

摩擦	substep	判定	いちばん高い粒	計算の秒	substep 1 との比
1.0	1	暴れた	5.2790	2.82	1.0
1.0	2	暴れた	1.8878	3.31	1.2
1.0	4	暴れた	7.4744	6.55	2.3
1.0	8	静か	0.1494	13.16	4.7
0.25	1	暴れた	13.5952	5.36	1.0
0.25	8	静か	0.0404	33.85	6.3
0.25	16	静か	0.0435	53.38	10.0

わかったこと

- ・ 暴れに「摩擦の境目」はなかった。摩擦 0.25~1.0

の9通りは、すべて60フレームまでに暴れた。違うのは始まる時刻で、摩擦 0.25 で F46、0.5 で F30、1.0 で F28。

- ・ 実験068の「摩擦 0.25 では暴れない」は、40フレームで止めていたからだった。同じ組み方で F46 から飛び始める。068の表の数字（40フレーム目の値）は正しいが、「暴れない」とは言えない。振り返り点検で068に注記を入れる。

- ・ 強い摩擦（1.5 / 2.0 / 3.0）と、摩擦 0 では暴れなかった。120フレームまで、いちばん高い粒は 0.2 以下。暴れるのは 0 と 1.5

の間だけで、摩擦を上げるほど悪くなるわけではない。なぜこの範囲だけなのかは確かめていない。

- ・ 飛ぶのは 547粒のうち 1~12粒。高さの平均は 0.03~0.13

のままで、液そのものは地面に広がっている。いちばん高い粒は 20 近くまで上がり続ける。平均だけを見ていたら気づかない（実験068の教訓どおり、平均と最大を両方見る）。

- ・ substep 8 で消える。摩擦 1.0 で substep 1 / 2 / 4 はまだ暴れ、8 で静かになった。摩擦 0.25 も substep 8・16 で120フレームまで静か。2 や 4 では足りない。4 では最大 7.47 と、1 のとき（5.28）より高く飛んだ。

- ・ ただし重い。60フレームで 2.82秒 → 13.16秒（4.7倍）、120フレームで 5.36秒 → 33.85秒（6.3倍）、substep 16 なら 53.38秒（10倍）。実験064の「substep 8 が落とすところ」は、Liquid の暴れ止めとしても同じ値になった。

- ・ 摩擦 0 では、液がわずかに地面より下へ沈んでいく。120フレーム目の高さの平均 -0.0443、いちばん高い粒でも -0.0412。substep 8 の摩擦 0.25 でも平均 -0.0158。飛ぶ暴れとは別の、小さなずれとして残る。

- ・ 実用の結論。Liquid を地面に置いたまま長く回すなら、摩擦は 0 か 1.5 以上にするか、substep を 8 にする。40フレームほどの短い場面では気づかないので、最後まで回して「いちばん高い粒」を確かめる。

- ・ 測り方について。最初は「摩擦の境目を挟み撃ちで探す」つもりだったが、60フレームの時点で全部が暴れたので問いを変えた。見立てどおりの手順を最後まで続けるより、出た数字に合わせて問いを直すほうが早い。

次にやること

- ・ 摩擦 1.5 以上ではなぜ暴れないのか（止まるまでの時間との関係）

- ・ 粒の細かさ（particlesep）を変えると、暴れ始める時刻は動くか

- ・ コライダの sticky も、組み込みの地面の sticky と同じか

sticky は天井にぶら下げられるか — 地面とコライダで効き方は同じ。ただし重さに逆らう力は無い

2026-09-24

何を作ったか

実験067で、組み込みの地面の

sticky（粘着）は「離れるとき」にだけ効くと分かった。落として跳ねさせると、戻る高さが sticky 1.0 で半分になった。実験069で、コライダの摩擦は組み込みの地面と同じものだった。では sticky も同じか。

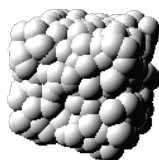
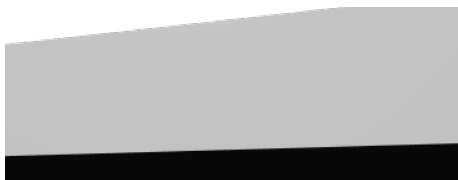
同じ落とし方（ $y = 2.5$ から、摩擦 0.25）で、地面とコライダの板に同じ sticky を入れて並べた。どちらも sticky を上げると戻る高さが減り、1~2 で頭打ちになる。地面は 0.026922 → 0.013596、コライダは 0.020205 → 0.013596。sticky 5 と 20 では、地面とコライダの値が6桁まで完全に一致した。途中の値は少し違うが、行き着く先は同じ。

「離れるときに引き止める」なら、天井にもぶら下げられるはず。天井の板の真下に塊を接して置き、sticky を 0 から 1000 まで上げた。塊はすべて落ちた。40フレーム目で下がった量は、sticky 0 で 13.01、1000 で 12.87。自由落下の 12.95 とほとんど変わらない。塊を天井に食い込ませて置いても同じだった。

sticky は「離れる速さ」を少し削るだけで、重さに逆らって物を支える力ではない。跳ね返りが半分になったのも、離れる瞬間の速さが削られた結果と考えると、両方の結果が合う。

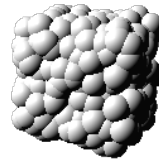
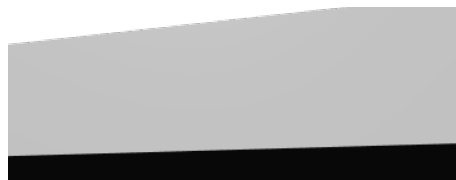
天井に接して置いた塊（0.46秒後）

天井の板の下面は $y = 3.0$ 。塊の一番上の粒は、最初のフレームで天井の 0.0615 下にある（粒の間隔 0.12 の半分）。2枚とも同じカメラ。



Houdini

sticky 0。12フレーム目までに 1.05 下がった。



Houdini

sticky 1000。1.01 下がった。ほとんど変わらない。

A. 落として跳ねさせる（ $y = 2.5$ から・摩擦 0.25）

「戻った高さ」は、いちばん低くなったあと、どこまで戻ったか（高さの平均で測る）。地面の値は実験067と6桁まで同じ。

sticky	地面：いちばん低い	地面：戻った高さ	コライダ：いちばん低い	コライダ：戻った高さ
0	0.310169	0.026922	0.310112	0.020205
0.25	0.310169	0.024870	0.310112	0.017774
0.5	0.310169	0.015851	0.310112	0.012788
1.0	0.310169	0.013617	0.310112	0.011758
2.0	0.310169	0.013596	0.310112	0.011881
5.0	0.310169	0.013596	0.310169	0.013596
20.0	0.310169	0.013596	0.310169	0.013596

B. 天井にぶら下げる（摩擦 0.25）

下がった量は、最初のフレームからの高さの平均の差。40フレーム（1.625秒）の自由落下は $\frac{1}{2} g t^2 = 12.9523$ 。

sticky	F6	F12	F24	F40	計算の秒
0	0.2227	1.0504	4.5435	13.0121	1.28
1	0.2106	1.0184	4.4716	12.8869	0.96
5	0.2099	1.0154	4.4640	12.8733	0.99
20	0.2099	1.0154	4.4640	12.8733	1.07
100	0.2099	1.0154	4.4640	12.8733	1.05
1000	0.2099	1.0154	4.4640	12.8733	0.99

C. 天井に食い込ませて置く（sticky 1000）

箱の上面を天井より上に出して置いた。どの置き方でも、最初のフレームで一番上の粒は天井の 0.0615 下にそろった。

箱の上面を天井より上に	F6	F12	F40	sticky 0 の F40
0	0.2075	1.0085	12.8454	13.0121
0.05	0.2060	1.0043	12.8285	13.0121
0.1	0.2049	1.0012	12.8162	13.0121
0.2	0.2026	0.9946	12.7896	13.0121

わかったこと

- ・コライダの sticky も、地面の sticky と同じ向きに効く。上げるほど跳ね返りが減り、地面は 0.026922 → 0.013596、コライダは 0.020205 → 0.013596。

- ・どちらも 1～2 で頭打ちになる。地面は sticky 2 以上で 0.013596 のまま。コライダも 5 以上で変わらない。sticky 5 と 20

では、地面とコライダの値が6桁まで完全に一致した（いちばん低い高さ 0.310169 も同じ）。

- ・途中の値は少し違う。sticky 0 で戻った高さは地面 0.026922、コライダ 0.020205。コライダの板のほうが少し跳ねにくい。いちばん低い高さも 0.310169 対 0.310112 と、0.00006 だけ違う。

- ・ sticky では天井にぶら下がれない。sticky を 1000 にしても、40フレームで 12.87 下がった。sticky 0 の 13.01、自由落下の 12.95 とほとんど変わらない。エラーも警告も出ない。

- ・ 天井に食い込ませても同じ。箱の上面を天井より 0.2 上に出して置いても 12.79 下がった。置き方によらず、最初のフレームで一番上の粒は天井の 0.0615 下にそろろう。

- ・ sticky が効くのは、離れる最初の一瞬だけ。sticky 0 と 1000 の下がった量の差は、F6 0.0128 / F12 0.0350 / F24 0.0795 / F40 0.1388。F12 から先は1秒あたり 0.089 ずつ一定の割合で開く。つまり最初の一瞬で速さに 0.089 の差ができ、そのあとは同じ加速で落ちている。支え続ける力ではなく、離れる瞬間の速さを削る数字と見るのが、跳ね返り（実験067）とぶら下がりの両方に合う。

- ・ 実用の結論。MPM で物を天井や壁に貼り付けたいなら、sticky ではできない。跳ね返りを抑える目的なら、コライダでも地面でも 1~2 で足りる（それ以上は変わらない）。

- ・ 計算の時間は、555粒・40フレームで 1回 0.7~2.7秒。

次にやること

- ・ 壁に向かって投げたとき、sticky は跳ね返りをどれだけ削るか
- ・ 粒の細かさ（particlesep）を変えると、実効の摩擦の上限（実験071）は動くか
- ・ 摩擦 1.5 以上の Liquid はなぜ暴れないのか

摩擦の上限は、粒の細かさで上がる — substep を 8 にしても1ミリも動かない

2026-09-24

何を作ったか

実験071で、回る台に塊を乗せたとき、滑り出す境目の $\omega^2 r$ が 3.1~3.7 にしかなかった。摩擦 1.0 なら 9.81 のはずで、摩擦が 0.3~0.4 ぶんしか効いていない。摩擦を 2.0 に上げて変わらなかった。この上限はどこから来るのか。計算の粗さから来るなら、細かくすれば上がるはず。

071と同じ条件（半径 2・摩擦 1.0・1辺 0.6 の塊）で、粒の間隔 `particlesep` を 0.18 / 0.12 / 0.08 / 0.06 / 0.04 と変え、境目の ω を挟み撃ちで探した。境目の $\omega^2 r$ を g で割った「実効の摩擦」は、0.42 / 0.34 / 0.65 / 0.73 / 0.79。0.12 より細かくすると、はっきり上がっていく。0.04 では式の境目 2.215 に対して 1.95~1.975 まで来た。

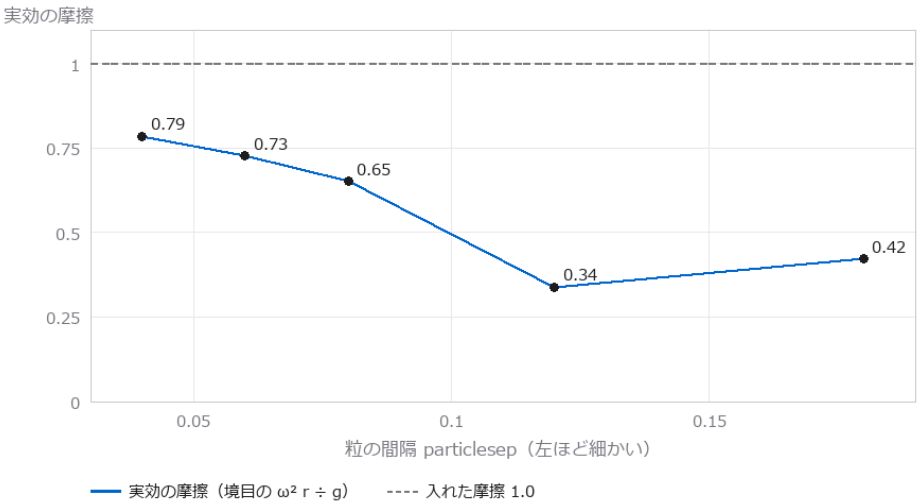
一方、`particlesep` 0.12 のまま `substep` を 1 / 4 / 8 と変えても、境目は 1.275~1.300 でまったく同じだった。挟み撃ちの6回の判定がすべて同じになった。時間だけが 3.48秒 → 10.93秒 → 17.58秒と5倍に増えた。

細かくするほうは安い。1回あたり 0.12 で 4.04秒、0.04 で 5.37秒。塊の粒は 108 から 3,352 へ31倍になったのに、時間は1.3倍しか増えなかった。

摩擦の上限は、1フレームを何回に分けて解くかではなく、粒と格子の細かさで決まっていた。

粒の間隔と実効の摩擦

実効の摩擦 = 境目の $\omega^2 \times$ 半径 $\div 9.81$ 。入れた摩擦は 1.0。点の数字は各間隔の実効の摩擦。



0.12 より細かくすると上がっていき、0.04 で 0.79。まだ 1.0 には届かない。0.18 が 0.12 より高いのは、塊が3粒ほどの幅しかないほど粗いため（理由は確かめていない）。

A・C. 粒の間隔を変える（半径 2・摩擦 1.0・substep 既定）

3秒で半径が 1.0 以上外へ動いたら「滑り出した」。6回の挟み撃ちで幅 0.025 まで詰めた。式の境目は ω 2.2147。

particlesep	塊の粒	境目の ω	実測 $\div$ 式	$\omega^2 r$	実効の摩擦	1回の秒
0.18	28	1.425~1.450	0.6491	4.1328	0.4213	3.42

particlesep	塊の粒	境目の ω	実測 ÷ 式	$\omega^2 r$	実効の摩擦	1回の秒
0.12	108	1.275~1.300	0.5813	3.3153	0.3380	4.04
0.08	403	1.775~1.800	0.8071	6.3903	0.6514	4.71
0.06	962	1.875~1.900	0.8523	7.1253	0.7263	5.48
0.04	3352	1.950~1.975	0.8861	7.7028	0.7852	5.37

B. substep を変える (particlesep 0.12 ・ 半径 2 ・ 摩擦 1.0)

3つとも挟み撃ちの判定が6回すべて同じになり、境目が完全に一致した。 ω 1.4 で外へずれた量だけ 2.96 / 3.20 / 3.16 と少し違う。

substep	境目の ω	実効の摩擦	1回の秒	substep 1 との比
1 (既定)	1.275~1.300	0.3380	3.48	1.0
4	1.275~1.300	0.3380	10.93	3.1
8	1.275~1.300	0.3380	17.58	5.1

わかったこと

- ・ 摩擦の上限は、粒を細かくすると上がる。実効の摩擦は particlesep 0.12 で 0.34、0.08 で 0.65、0.06 で 0.73、0.04 で 0.79。入れた摩擦 1.0 に近づいていく。
- ・ substep では1ミリも動かない。1 / 4 / 8 で境目が 1.275~1.300 のまま、挟み撃ちの判定6回がすべて同じだった。時間だけが 5.1倍になる。
- ・ 細かくするほうが安い。1回あたり 0.12 で 4.04秒、0.04 で 5.37秒。塊の粒は 108 → 3,352 と31倍なのに、時間は1.3倍。substep 8 の 17.58秒よりずっと速い。
- ・ 実験071の「摩擦は 0.3~0.4 ぶんしか効かない」は、particlesep 0.12 での話だった。MPM の摩擦そのものの限界ではなく、粒の粗さから来ていた。071の結論には、この条件を添える必要がある。
- ・ 0.04 でも、まだ式には届かない。境目は式の 0.886倍。どこまで細かくすれば 1.0 に届くのか、それとも届かないのかは、この範囲では決められない。
- ・ いちばん粗い 0.18 は 0.12 より高く出た (0.42 対 0.34) 。塊は1辺 0.6 なので、0.18 では3粒ほどの幅しかない。この粗さでは塊として振る舞っていない可能性があり、傾向の外に置いた。
- ・ 細かくすると、境目の下のずれも大きくなる。0.04 では ω 1.8 で 0.23、1.95 で 0.67 外へずれた (0.12 では ω 1.2 で 0.10)。境目の判定「1.0 以上」に近い値が増えるので、細かいほど境目はぼやける。
- ・ 実用の結論。MPM で摩擦を効かせたいなら、substep ではなく particlesep を細かくする。substep は Liquid の暴れ止め (実験073) には効くが、摩擦の効きには効かない。直したい症状で、回すつまみが違う。
- ・ 測り方について。071と同じ関数で境目を探し、つまみだけを変えた。particlesep 0.12 ・ substep 既定の条件は、071の B とこの実験の A ・ B で3回測ったことになり、境目は 1.25~1.30 / 1.275~1.300 / 1.275~1.300 と重なった。

次にやること

- ・ particlesep をさらに細かくして、実効の摩擦が 1.0 に届くか
- ・ 粒の細かさで、Liquid の暴れ始め (実験073) も動くか

・実験067・070の「強い摩擦ほど式から外れる」も、細かくすれば縮むか

粒を細かくしても、摩擦の式には戻らない — 細かいほど塊が潰れ、かえって式から離れる

2026-09-24

何を作ったか

実験067で、塊を地面に滑らせると、摩擦 0.25 までは式「止まるまでの距離 = $v^2 \div (2 \mu g)$ 」に乗ったが、摩擦を上げるほど式より遠くまで滑った。実験070の動く板でも、強い摩擦ほど式より短くしか運ばれなかった。どちらも粒の間隔は 0.12。

実験075で、回る台の摩擦の上限は、粒を細かくすると上がると分かった。では067・070の外れも、細かくすれば縮むか。粒の間隔を 0.12 / 0.08 / 0.06 / 0.04 と変えて、同じ場面を測り直した。

縮まなかった。逆に広がった。滑る塊（初速 4）の「実測 ÷ 式」は、摩擦 1.0 で 1.46 → 1.58 → 1.64 → 1.73、摩擦 2.0 で 2.82 → 3.14 → 3.31 → 3.36。動く板も 0.939 → 0.918 → 0.908 → 0.896 と、細かいほど式から遠ざかる。摩擦 0.25 だけは、どの細かさでも 1.000 前後（0.9996～1.0009）のまま。

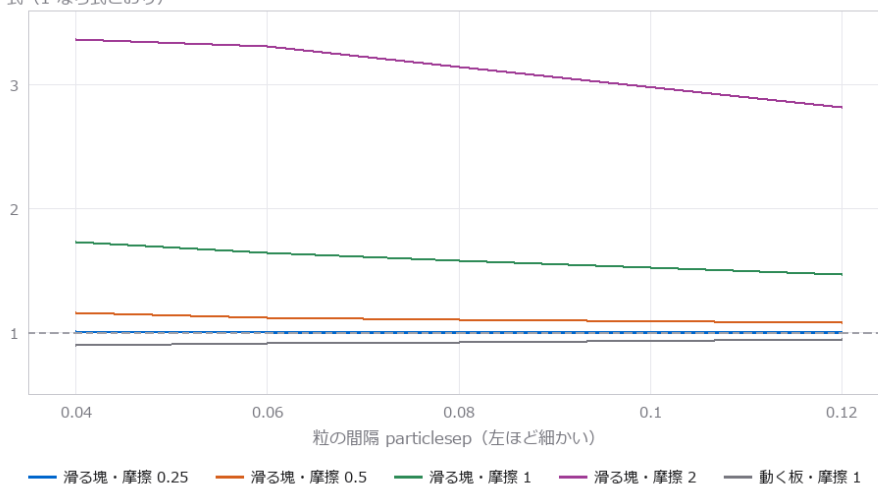
同時に、止まったあとの塊の高さの平均が下がっていく。摩擦 1.0 の滑る塊で 0.340 → 0.263 → 0.233 → 0.205。元は 0.5 なので、細かいほど大きく潰れている。実験068で「潰れるほど式から外れる」と分かったのと同じ筋で、細かくすると塊が柔らかく振る舞い、そのぶん外れが大きくなった。

075と逆の結果になったのは、見ているものが違うからと考えられる。075は「止まっていられるか」（滑り出す境目）で、細かいほど摩擦がよく効いた。067・070は「滑りながらどう止まるか」で、ここでは塊の潰れが効く。同じ「摩擦の外れ」でも、原因は1つではなかった。

粒の間隔と「実測 ÷ 式」

1 なら式どおり。滑る塊は 1 より上（式より遠くまで滑る）、動く板は 1 より下（式より短く運ばれる）に外れる。どちらも、細かいほど 1 から離れる。

実測 ÷ 式（1 なら式どおり）



摩擦 0.25 だけは、どの細かさでもほぼ 1。摩擦 0.5 以上は、細かくするほど外れが大きくなる。

A. 滑る塊（1辺 1.0・初速 4・40フレーム。実験067と同じ組み方）

間隔 0.12 の行は、実験067の値と6桁まで同じ。「高さ」は40フレーム目の高さの平均で、元は 0.5。

間隔	粒	摩擦	滑った距離	式	実測 ÷ 式	止まった秒	高さ
0.12	547	0.25	3.260808	3.261978	0.999642	止まらず	0.5045
0.12	547	0.5	1.751774	1.630989	1.074056	1.000	0.3551
0.12	547	1.0	1.194665	0.815494	1.464958	0.875	0.3402
0.12	547	2.0	1.148402	0.407747	2.816457	0.833	0.3250
0.08	1914	0.25	3.262817	3.261978	1.000257	止まらず	0.5003
0.08	1914	0.5	1.792811	1.630989	1.099217	0.958	0.2926
0.08	1914	1.0	1.284540	0.815494	1.575167	0.917	0.2626
0.08	1914	2.0	1.282087	0.407747	3.144319	0.917	0.2503
0.06	4580	0.25	3.264006	3.261978	1.000622	止まらず	0.4992
0.06	4580	0.5	1.819939	1.630989	1.115850	1.042	0.2629
0.06	4580	1.0	1.336022	0.815494	1.638297	1.000	0.2325
0.06	4580	2.0	1.348446	0.407747	3.307065	0.917	0.2254
0.04	15591	0.25	3.264953	3.261978	1.000912	止まらず	0.4954
0.04	15591	0.5	1.881290	1.630989	1.153466	1.125	0.2216
0.04	15591	1.0	1.409456	0.815494	1.728345	1.042	0.2045
0.04	15591	2.0	1.370689	0.407747	3.361615	1.042	0.2102

B. 動く板（板の速さ 4・摩擦 1.0・40フレーム。実験070と同じ組み方）

式の距離は 5.684506。間隔 0.12

の行は実験070の値と6桁まで同じ。計算の秒は、最初と最後のフレームだけ測った場合の値。

間隔	粒	運ばれた距離	実測 ÷ 式	最後の vx	高さ	計算の秒
0.12	547	5.336132	0.938715	3.998951	0.3382	2.03
0.08	1914	5.219249	0.918154	4.000077	0.2633	2.04
0.06	4580	5.164202	0.908470	3.999630	0.2314	2.70
0.04	15591	5.091048	0.895601	4.000787	0.2055	4.05

わかったこと

- ・粒を細かくしても、摩擦の式には戻らない。むしろ離れる。滑る塊の摩擦 1.0 で「実測 ÷ 式」が 1.46（間隔 0.12）→ 1.73（0.04）、摩擦 2.0 で 2.82 → 3.36。動く板は 0.939 → 0.896。
- ・摩擦 0.25 だけは、どの細かさでも式どおり。0.999642 / 1.000257 / 1.000622 / 1.000912。弱い摩擦で塊が形を保ったまま滑る場面では、細かさは関係ない。
- ・細かいほど、塊が潰れる。摩擦 1.0 の滑る塊で、止まったあとの高さの平均が 0.340 → 0.263 → 0.233 → 0.205（元は 0.5）。動く板でも 0.338 → 0.206。実験068の「潰れるほど式から外れる」と同じ筋で、潰れが大きくなるぶん外れも大きくなった。
- ・強い摩擦でも、止まる時間はほとんど縮まない。式では摩擦 1.0 で 0.41秒、2.0 で 0.20秒で止まるはずが、実測は 0.83～1.04秒。摩擦 1.0 と 2.0 で滑った距離もほぼ同じ（間隔 0.04 で 1.409 と 1.371）。

- ・実験075と逆の向きになった。075（回る台で止まっていられるか）では、細かいほど摩擦がよく効いた。こちら（滑りながらどう止まるか）では、細かいほど式から離れた。同じ「摩擦が式に合わない」でも、原因は1つではない。一方は粗さ、一方は潰れと考えると両方に合うが、潰れの中身は確かめていない。
- ・粒の間隔 0.12 の値は、実験067・070と6桁まで同じだった。同じ組み方を別のスクリプトから呼んでも、結果は変わらない。
- ・計算の時間について。動く板（B）では、粒が 547 → 15,591 と28倍になっても、時間は 2.03秒 → 4.05秒の2倍だった。滑る塊（A）は 2.2秒 → 20秒と9倍になったが、A は測るために毎フレーム全部の粒を読んでいて、その時間が大きく乗っている。計算の重さを比べるときは、測る処理を最後のフレームだけにする。
- ・実用の結論。MPM の塊を「式どおりに滑って止まる」ようにしたいなら、摩擦は 0.25 程度にとどめる。摩擦を上げて止まる時間はほとんど縮まず、細かくするとかえって外れる。

次にやること

- ・塊の材質を固く（Elastic）すると、細かくしても潰れず式に戻るか
- ・Liquid の暴れ始め（実験073）は、粒の細かさで動くか
- ・測る処理の重さ：毎フレーム全部の粒を読むと、どれだけ時間が乗るか

液体の暴れは、粒を細かくしても消えない — 飛ぶ高さは下がるが、始まる時刻は変わらない

2026-09-24

何を作ったか

実験073で、Liquid を摩擦のある地面に置くと、しばらくして数粒だけが飛び始めた（粒の間隔 0.12 で、摩擦 1.0 は F28、0.5 は F30 から）。substep 8 で消えたが、5倍前後かかった。実験075では、摩擦の上限は substep では動かず、粒の細かさで動いた。では液体の暴れはどうか。

摩擦 1.0 と 0.5 で、粒の間隔を 0.16 / 0.12 / 0.08 / 0.06 と変え、120フレームまで回した。値は実験077で作った「まとめて読む」関数で毎フレーム読んでいる。

暴れは、どの細かさでも起きた。始まるフレームは、摩擦 1.0 で F27 / F28 / F22 / F26、摩擦 0.5 で F32 / F30 / F40 / F33。細かさとの決まった関係は見えず、F22~F40 の間に散らばる。

変わったのは飛ぶ高さだった。摩擦 1.0 で、120フレームまでのいちばん高い粒は 46.3 / 19.7 / 11.2 / 4.4。細かいほど低い。ただし飛ぶ粒の数は、どの細かさでも数粒~41粒で、0 にはならない。

間隔 0.12 の2行は、実験073の値と一致した（摩擦 1.0 で F28・最後のいちばん高い粒 19.6697、摩擦 0.5 で F30）。

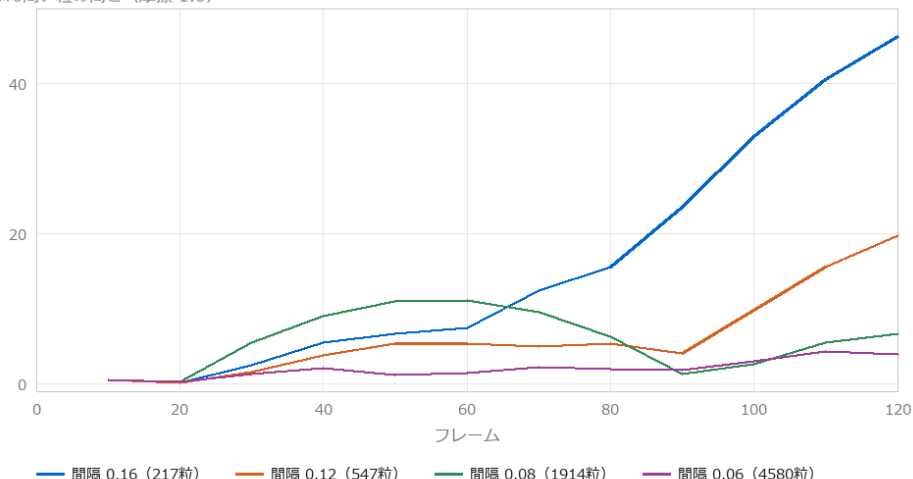
液体の暴れを止めるつまみは

substep（実験073）で、粒の細かさではない。実験075の摩擦とは、効くつまみが逆だった。

いちばん高い粒の高さ（摩擦 1.0・10フレームおき）

1辺 1.0 の Liquid の箱を、初速 4 で組み込みの地面に滑らせた（実験068・073と同じ組み方）。

いちばん高い粒の高さ（摩擦 1.0）



どの細かさでも F20~30 から上がり始める。細かいほど低いところにとどまるが、0.06 でも 4.4 まで上がる。

粒の間隔ごとの暴れ（120フレーム）

「始まり」は、いちばん高い粒が初めて 1.0

を超えたフレーム（6フレーム目以降）。「飛んでいる粒」は、F30

以降に10フレームおきに数えた、高さ 0.5 より上の粒の最大。摩擦 0.5・間隔 0.08

は、117~120フレーム目で計算結果が空になった（理由は確かめていない）。

摩擦	間隔	粒	始まり	いちばん高い粒	飛んでいる粒	割合	計算の秒
1.0	0.16	217	F27	46.274	5	2.3%	3.38
1.0	0.12	547	F28	19.670	5	0.91%	2.77
1.0	0.08	1914	F22	11.208	1	0.05%	3.40
1.0	0.06	4580	F26	4.380	7	0.15%	4.71
0.5	0.16	217	F32	37.832	5	2.3%	3.70
0.5	0.12	547	F30	11.769	7	1.28%	3.88
0.5	0.08	1914	F40	8.012	23	1.2%	4.68
0.5	0.06	4580	F33	3.725	41	0.9%	5.81

わかったこと

- ・液体の暴れは、粒を細かくしても消えない。間隔 0.16～0.06 の8通りすべてで、数粒が飛び始めた。
- ・始まる時刻は、細かさで決まらない。摩擦 1.0 で F22～F28、摩擦 0.5 で F30～F40。細かくして早まるとも遅くなるとも言えない。
- ・飛ぶ高さは、細かいほど低い。摩擦 1.0 で 46.3 → 19.7 → 11.2 → 4.4、摩擦 0.5 で 37.8 → 11.8 → 8.0 → 3.7。ただ 0.06 でも床から 4 近く飛ぶ粒があり、見た目では気づく。
- ・飛ぶ粒の数は、どの細かさでも一握り。1～41粒（全体の 0.05～2.3%）。平均の高さはほとんど動かないので、平均だけ見ていると気づかない（実験068・073と同じ）。
- ・止めるつまみは substep。実験073で substep 8 にすると消えた。粒の細かさでは消えない。実験075の摩擦の上限（粒の細かさで動き、substep では動かない）とは、効くつまみがちょうど逆だった。
- ・間隔 0.12 の値は、実験073と一致した。摩擦 1.0 で始まり F28、120フレーム目のいちばん高い粒 19.6697。摩擦 0.5 で始まり F30。
- ・摩擦 0.5・間隔 0.08 では、117～120フレーム目で計算結果が空になった。エラーの表示は取れていない。暴れと関係があるかは確かめていない。
- ・計算の時間は 120フレームで 2.77～5.81秒。粒が 217 → 4,580 と21倍になっても、時間は1.5～2倍ほど。値は毎フレーム読んだが、実験077の「まとめて読む」やり方なので、読む時間はほとんど乗っていない。
- ・測り方について。最初は「0.5 より上の粒の最大」をフレーム1から数えていたが、置いたばかりの箱は上半分が 0.5 より上にあるので、その数（217～4,580粒の約半分）を拾っていた。数える前に、その量が最初からいくつあるかを確かめる。F30 以降だけで数え直した。

次にやること

- ・ substep 8 が要るのは、どの摩擦・どの細かさでも同じか
- ・計算結果が空になるフレームは、何が起きているのか
- ・ 030・060に続く振り返り点検（061～090）を 090 で行う

mpmsource は箱に「体積 ÷ Particle Separation³」個の粒を詰める — pscale には Separation そのものが入る

2026-09-24

何を作ったか

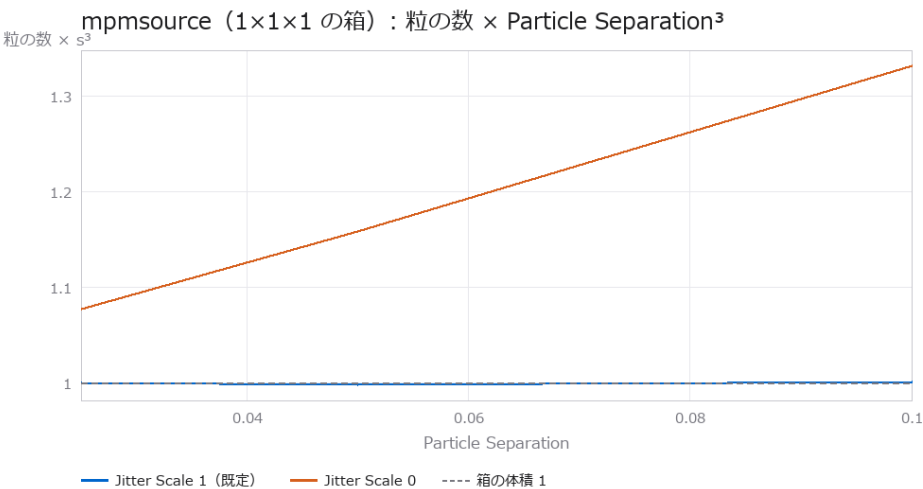
1 × 1 × 1 の箱を mpmsource (入力 2 に mpmcontainer) で粒にし、mpmcontainer の Particle Separation s と、mpmsource の Jitter Scale を変えた。

****既定 (Jitter Scale 1) では、粒の数 × s^3 が箱の体積 1 にほぼ一致した**** ($s = 0.1$ で $1.0010 \cdot s = 0.05$ で $0.9980 \cdot s = 0.025$ で 0.9993)。粒1つが s^3 の体積を受け持つ。

****Jitter Scale 0 だと、粒は升目どおりに並び、箱の面の上にも乗る。数値は $(1/s + 1)^3$ と3通りとも一致 ($s = 0.1$ で $1,331 = 11^3$)。x の並びは -0.5 から 0.5 まで 11 層。そのぶん、粒の数 × s^3 は体積より多くなる (実験141 の pointsfromvolume の Grid と同じ)。**揺らぎ (Jitter) を入れると、面の上に並ぶぶんが消えて数が合う。

****粒には pscale と density が付き、mass は付かない。pscale は Particle Separation と同じ値 ($s = 0.05$ なら 0.05 。半径なら半分のはずだが、そうではない)、density はどの粒も既定の 400。**

Particle Separation・Jitter と粒



Jitter 1 (青) は 1 に乗る。Jitter 0 (橙) は面の上の分だけ多い。

Separation	Jitter Scale	粒の数	粒の数 × s^3	$(1/s+1)^3$	pscale	density	x の層	秒
0.1	1	1,001	1.00100	1,331	0.1	400	—	0.306
0.1	0	1,331	1.33100	1,331	0.1	400	11	0.173
0.05	1	7,984	0.99800	9,261	0.05	400	—	0.214
0.05	0	9,261	1.15763	9,261	0.05	400	21	0.198
0.025	1	63,956	0.99931	68,921	0.025	400	—	0.242
0.025	0	68,921	1.07689	68,921	0.025	400	41	0.143

わかったこと

- ・ 粒の数は 体積 $\div$ Separation³。Separation を半分にすると粒は 8 倍（重さの見積もりに使える）。
- ・ pscale は Separation と同じ値。粒を球で見せるとき、そのまま使うと隣と重なる。
- ・ Jitter を 0 にすると、面の上にも粒が並ぶ。数が $(1/s + 1)^3$ に増える。

次にやること

- ・ 丸い形（球）を詰めたときの数
- ・ mpsolver で落としたときに粒の数と体積が保たれるか

FLIP のダムブレイクは Particle Separation 0.04 で足りる — 0.06 より粗いと壁ぎわの数粒が大きな塊に見え、水も 1~2 割増える

2026-09-24

何を作ったか

****課題:** 水の柱が崩れて流れる場面で、FLIP の粒の間隔 (Particle Separation) をどこまで粗くしても見た目が保てるか。時間はどれだけ減るか。 ******

幅 3 × 高さ 1.5 × 奥行き 1 の水槽の左端に、幅 0.6 × 高さ 0.9 の水 (体積 0.54) を置いて崩した。粒の間隔を 0.02 (基準) ・ 0.03 ・ 0.04 ・ 0.06 ・ 0.08 と変えて 60 フレーム (24 fps で 2.5 秒) 回し、波の到着・液面の形・面にしたときの体積・時間を測り、同じカメラで撮った。

****結論:** この大きさなら 0.04。 ****波が右の壁に届くのは 0.02~0.06 でどれもフレーム 17 (0.08 だけ 19)。** 落ち着いた液面 (フレーム60) の形の差は、0.02 と比べて 0.03 で 0.072、0.04 で 0.068、0.06 で 0.229、0.08 で 0.119 (高さの二乗平均の差、単位は水槽と同じ)。面にした水の体積はフレーム60で 0.02 0.472 (-13%)、0.04 0.568 (+5%)、0.06 0.592 (+10%)、0.08 0.671 (+24%)。

****0.02 で面の体積が 13%**

少ないのは、面にする段階で痩せているため。 ****粒の液面の高さ (区間ごとの粒の y の 99 パーセンタイル) の平均から出した水の量は、0.02 のフレーム60で 0.540 と、入れた 0.54 に合っていた。** 粒は水を保っていて、particlefluidsurface (既定の設定) が細かい間隔で水を薄く面になっている。

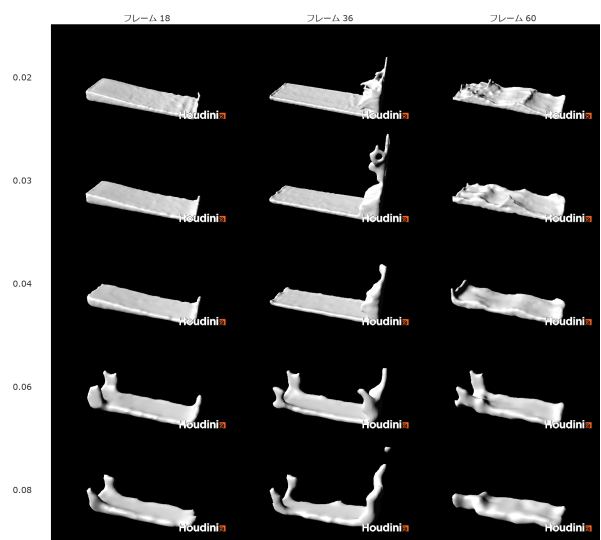
****粗くしても時間はあまり減らない。 ****60 フレームで 0.02 が 13.6 秒、0.04 が 9.1 秒、0.08 が 6.1 秒。粒は 76,261 個から 1,123 個へ約 68 分の 1 になるのに、時間は 2.2 分の 1 にしかならない。理由を切り分けた。水を入れない空の水槽でも 1.8~2.3 秒かかる。さらに、1 スレッドに絞る (hython -j 1) と 0.02 は 50.5 秒、0.08 は 5.5 秒で、差は 9.2 倍に開いた。この機械 (32 スレッド) では、粒の多い細かい間隔ほど並列で速くなり、粗い間隔は並列の恩恵をほとんど受けない。時間の差が縮むのはこのため。****

****0.06 より粗いと、左の壁に水の柱が残ったように見える。 ****原因を切り分けた。壁ぎわの高い所 ($x < -1.3$, $y > -0.45$) にいる粒は、0.06 のフレーム60で全体の 0.2% (約 6 個) だけ。0.04 では 0.0% (フレーム36)。壁を 0.2 から 0.6 に厚くしても変わらず (0.22%)、最初の水を壁から 0.1 離すと減った (フレーム36で 0.12%)。壁の衝突が効いていないのではなく、壁ぎわに残った数粒が、粒の間隔が粗いぶん大きな半径で面になり、塊に見えている。****

****粒の数が減るのは、水が消えたからではない。 ****0.06 では粒が 60 フレームで 2,908 → 1,966 個 (68%) に減った。壁を 0.85 から 1.45 に高くしても同じ減り方 (68%) で、壁を越えて外へ出たのではない。既定で入っている Reseeding を切ると 2,908 個のまま変わらず、面の体積は 0.583 (入れたままは 0.592)。Reseeding が混んだ所の粒を間引いているだけで、見た目の水の量はほぼ同じ。****

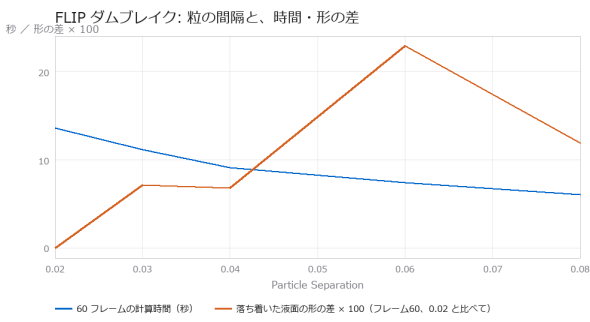
****しぶきの高さは、どの間隔でも揃わない。 ****右の壁を駆け上がるいちばん高い所は 0.02 で 0.61・0.03 で 1.26・0.04 で 0.81・0.06 で 0.61・0.08 で 1.17。細かくするほど一方向に近づくのではなく、行ったり来たりする。しぶきの先端は、間隔を変えるたびに別物になると考えてよい。****

同じカメラで撮った水（上から 0.02・0.03・0.04・0.06・0.08）

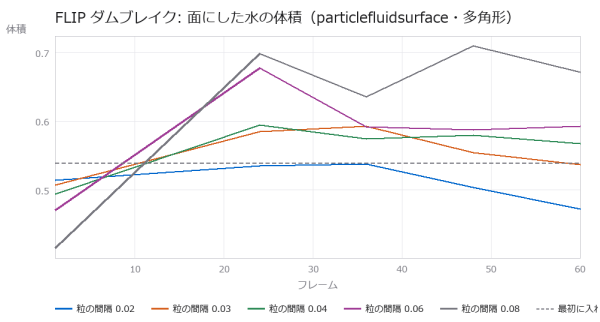


0.06・0.08 は左の壁ぎわに塊が残る。0.04 までは平らに広がる。

粒の間隔と、時間・形・体積



時間（青）は粗くしてもゆっくりしか減らない。形の差（橙）は 0.06 で跳ね上がる。



粗いほど面にした水が増える（点線が入れた水 0.54）。

粒の間隔	粒の数（最初）	粒の数（F60）	壁に届くフレーム	形の差（F48）	形の差（F60）	体積（F60）	計算 秒	面にする 秒
0.02	76,261	67,801	17	0.000	0.000	0.472	13.64	0.21
0.03	23,719	22,987	17	0.119	0.072	0.537	11.14	0.17
0.04	10,103	10,106	17	0.101	0.068	0.568	9.09	0.13
0.06	2,908	1,966	17	0.282	0.229	0.592	7.41	0.12
0.08	1,123	582	19	0.249	0.119	0.671	6.08	0.11

原因の切り分け（粒の間隔 0.06 など）

条件	粒の数 F1 → F60	壁ぎわの粒の割合 F36	体積（F60）
0.06・壁の上端 1.45（基準）	2,908 → 1,966	0.18%	0.592
0.06・壁の上端 0.85	2,908 → 1,973	—	0.570
0.06・Reseeding 切	2,908 → 2,908	—	0.583
0.06・壁の厚み 0.6	—	0.22%	—

条件	粒の数 F1 → F60	壁ぎわの粒の割合 F36	体積 (F60)
0.06・水を壁から 0.1 離す	—	0.12%	—
0.04 (基準)	—	0.00%	0.568

わかったこと

- ・水槽 3 m・水 0.5 m³ くらいの場面なら、Particle Separation は 0.04。流れと落ち着いた形は 0.02 とほぼ同じで、時間は 1.5 分の 1。
- ・0.06 より粗くしても時間はあまり減らない（多コアの機械では特に）。代わりに、壁ぎわの数粒が塊になり、水が 1～2 割増えて見える。プレビューでも 0.04 で回したほうが得。
- ・粒の数が減っても、水が消えたとは限らない。Reseeding（既定で入）が間引いているだけ。水の量は粒の液面の高さか、面にした体積で見る。
- ・細かい間隔（0.02）では、面にすると水が 1 割ほど痩せる。粒は水を保っている。面の厚みが足りなく見えたら particlefluidsurface の設定を疑う。
- ・しぶきの先端の形は、間隔を変えると別物になる。プレビューのしぶきを本番でそのまま再現しようとしない。
- ・SOP の FLIP を一から組むときの落とし穴 4 つ。(1) 容器 (flipcontainer) は壁の無い開いた箱で、外に出た粒は消える。床と壁は flipcollide の 4 番目の入力につなぐ。(2) flipsource で Add pscale を入れないと「Required attribute pscale is missing」で止まる。(3) Volume Name を既定の surface のままにすると水がソルバーに入らなかった（粒 5 個）。source にし、最初のフレームだけ湧かせる。(4) その入れ方で Narrow Band（既定で入）のままだと、内側の粒が消えていく（0.08 で 12 フレームに 912 → 160 個）。
- ・訂正: 実験019
の「落ちる水」は、床の無い容器を落ち抜けていただけだった。今回と同じ組み方（flipcollide で床を足す）でないと、水は溜まらない。